Estruturas de Dados

Samuel da Costa Alves Basilio

samuelbasilio@leopoldina.cefetmg.br

Apresentação da Disciplina

Horários :

CMA06 – Teoria

Terça 10:40 – 12:20

CMA07 – Laboratório

Terça 17:00 – 18:40;

Carga horária semanal por disciplina:

2 h/aulas;

Carga horária total por disciplina :

30h/aulas;

Pré-requisitos

CMA01 - Programacão de Computadores I

CMA02 - Lab Programacão de Computadores I

- Estruturas de controle
  - Alternativa
  - Repetição
- Declaração e escopo de variáveis
- Subrotinas
  - Funções
  - Procedimentos
- Estruturas de dados homogêneas
  - Vetores numéricos
  - Vetores de caracteres
  - Matrizes

Objetivos Gerais

A disciplina deverá possibilitar ao estudante:

1. Familiaridade com as principais estruturas de dados e suas abstrações.
2. Capacidade de criar estruturas de dados com alocação estática e dinâmica de memória.
3. Capacidade de utilizar os conceitos relativos ao projeto e implementação de aplicações utilizando as estruturas de dados clássicas: listas, filas, pilhas e árvores.
4. Domínio dos conceitos relativos ao projeto, implementação e uso de algoritmos de pesquisa e ordenação.

Conteúdo da Disciplina

1. Conceitos introdutórios em estruturas de dados
2. Tipos abstratos de dados
3. Estruturas de dados: listas, pilhas, filas e árvores
4. Algoritmos de pesquisa
5. Algoritmos de ordenação

Principais Referências Bibliográficas

- ZIVIANI, N. Projeto de Algoritmos, com implementações em Pascal e C. Editora Pioneira, 4ª edição, 1999.

  Site do autor: http://www.dcc.ufmg.br/algoritmos-java/

- SILVA, R.L.S. e OLIVEIRA, A. M. Algoritmos em C , 1 edição. 2014

  Site do autor: https://sites.google.com/site/algoritmosufjf/material

- CORMEN, T. H. et.al. Algoritmos: teoria e prática, 3 edição. Ed. Campus, 2002.

  Site do autor: http://mitpress.mit.edu/algorithms/

Links para vídeo aulas

https://sites.google.com/site/algoritmosturmaespecial/

https://www.youtube.com/user/progdescomplicada

Método de Avaliação (previsão)

Avaliação 1
    10 pontos

Avaliação 2
    30 pontos

Avaliação 3
    30 pontos

Avaliação 4
    30 pontos

Total 100 pontos

Aproveitamento mínimo de 60%.
Presença mínima: 75%. (Em ambas disciplinas)

Dúvidas

- Dúvidas?
- Perguntas?
- Sugestões?
- Requisições?

REVISÃO: Tipos de Subrotinas

- procedimentos

```
void ImprimeSoma (int a, int b)
{
    printf("%d", a+b);
}
```

- funções

```
int Soma (int a, int b)
{
    return (a+b);
}
```

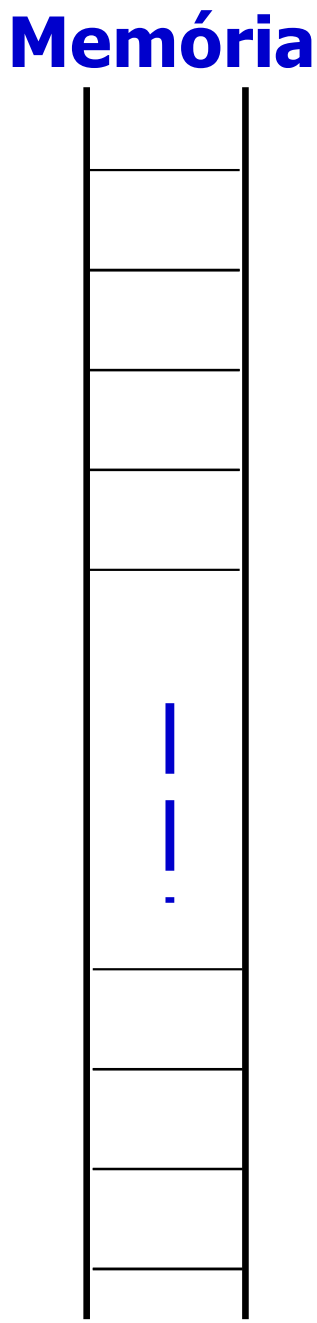
Parâmetros de uma subrotina em C Exemplo

Chamada por valor: Qualquer alteração feita no parâmetro não reflete em alteração no argumento.

Chamada por referência: Todas as alterações realizadas no parâmetro, refletem em alterações no argumento, ou seja, ambas as variáveis apontam para o mesmo endereço de memória. Para isso, é necessário que seja passado o endereço do argumento e o parâmetro receba-o na forma de ponteiro.

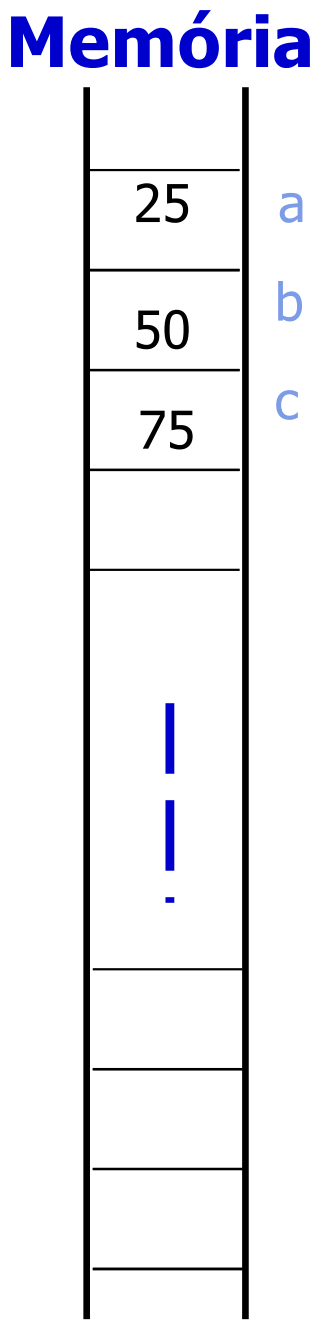
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



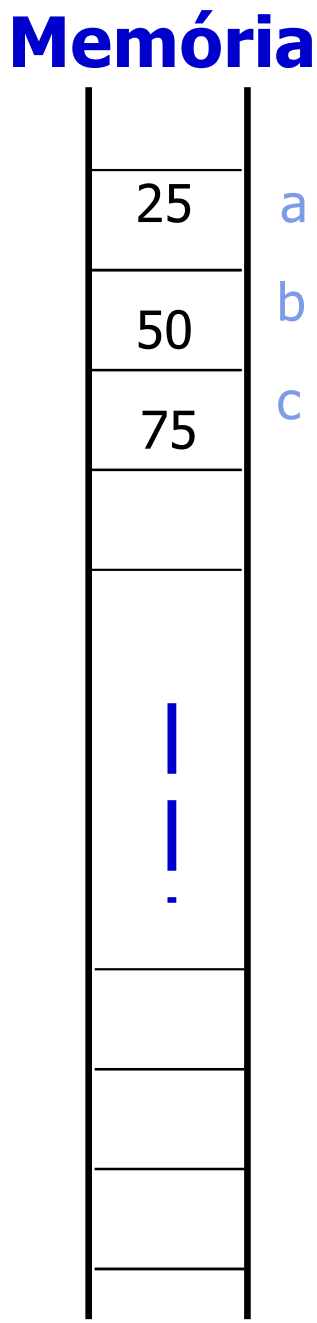
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



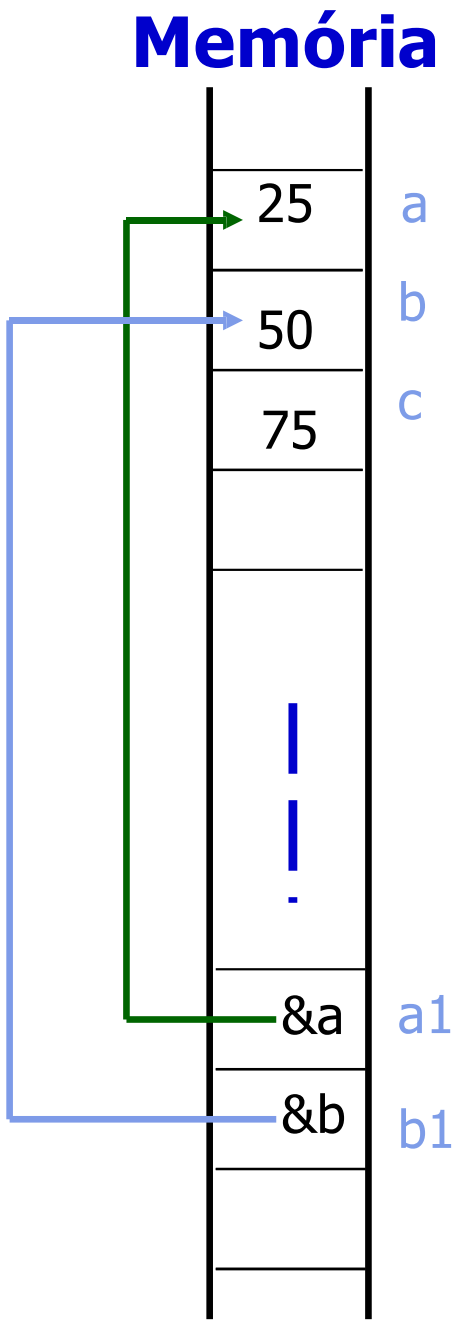
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



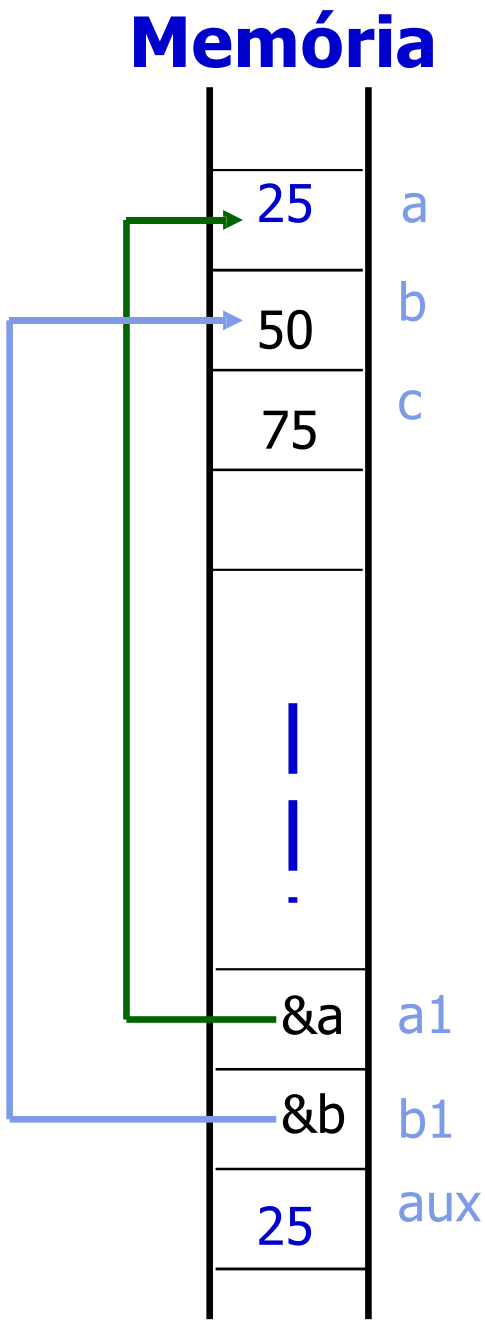
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
   int aux=*a1;
   *a1=*b1;
   *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



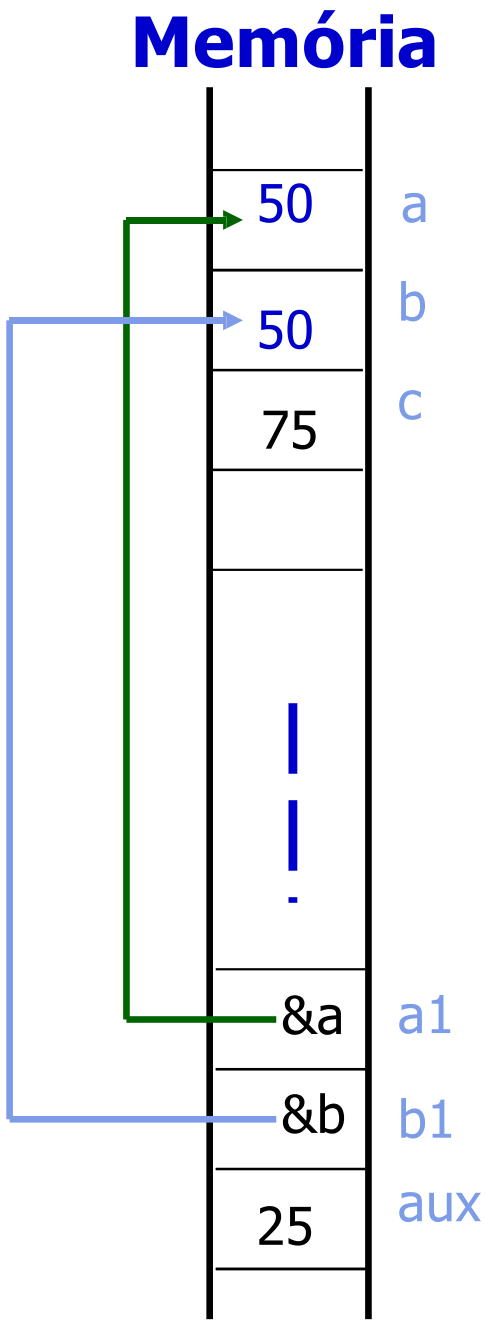
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



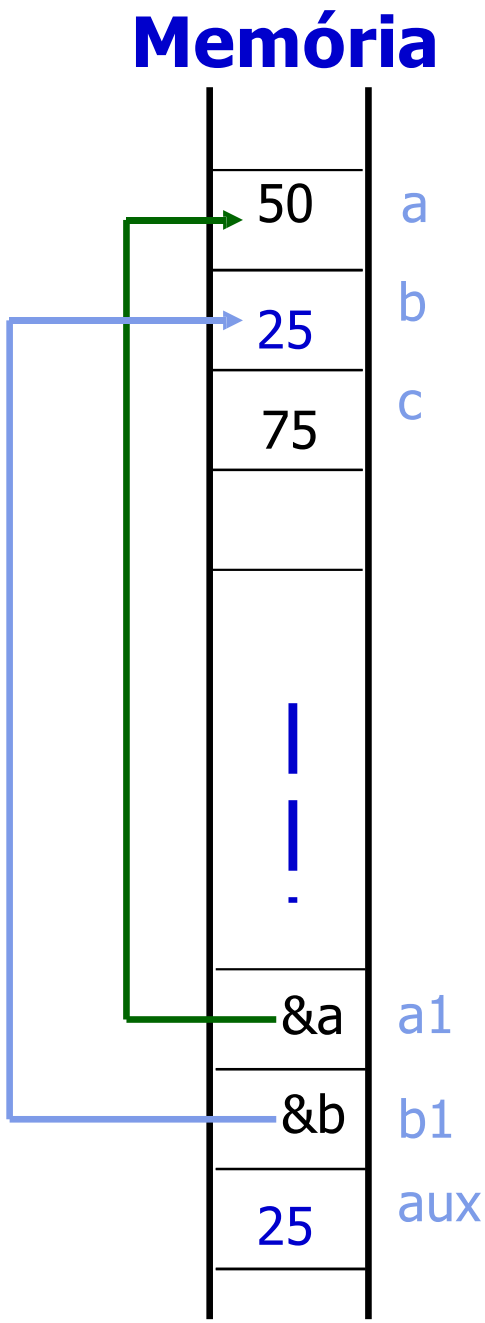
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



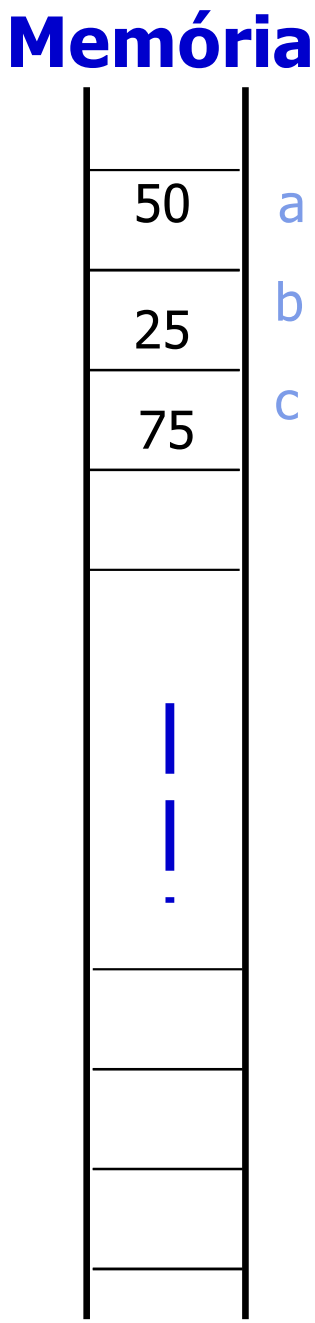
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



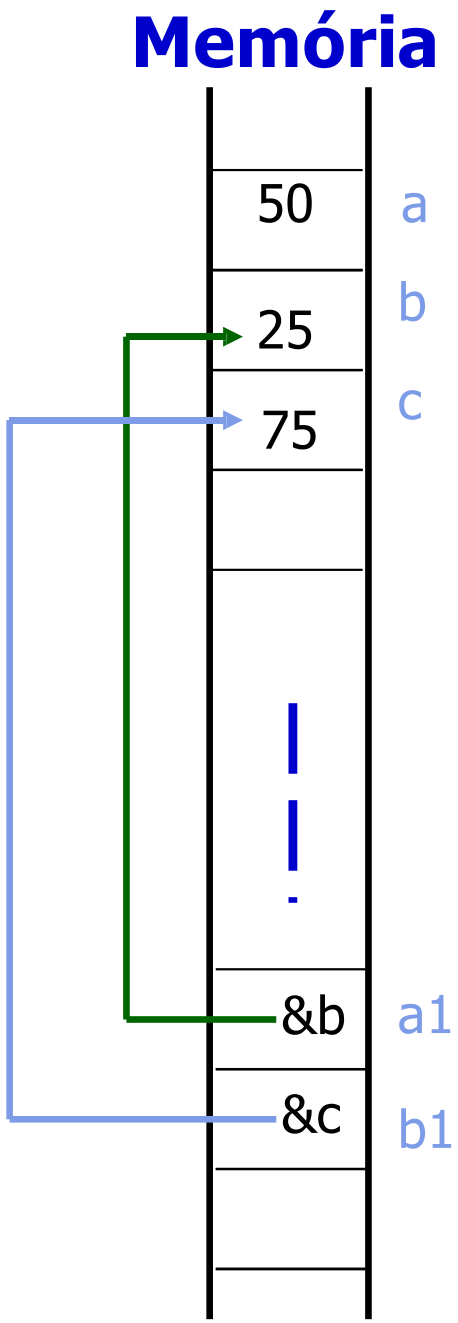
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



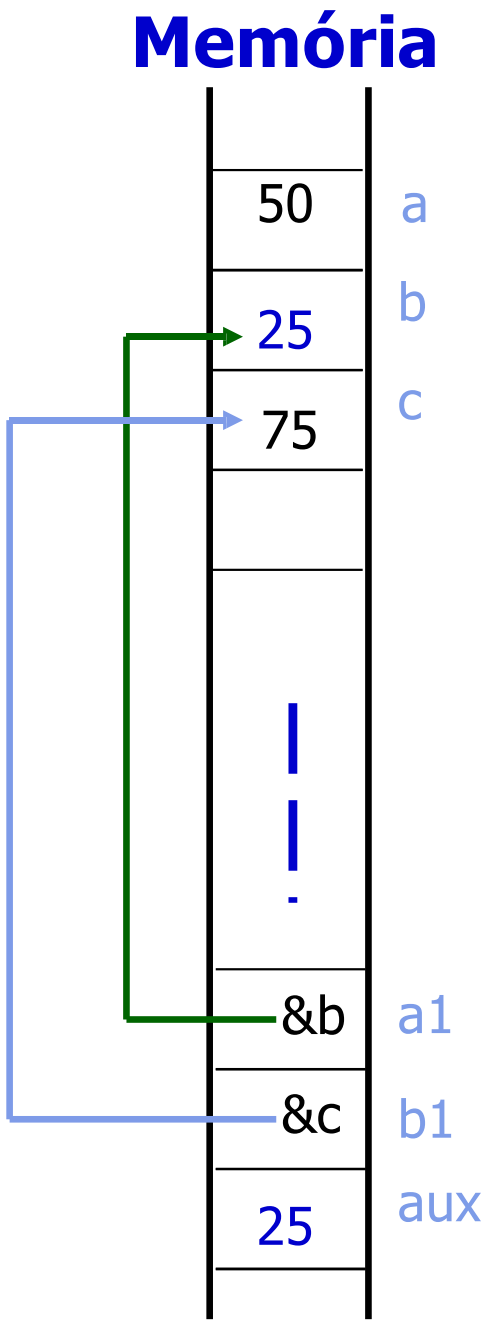
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



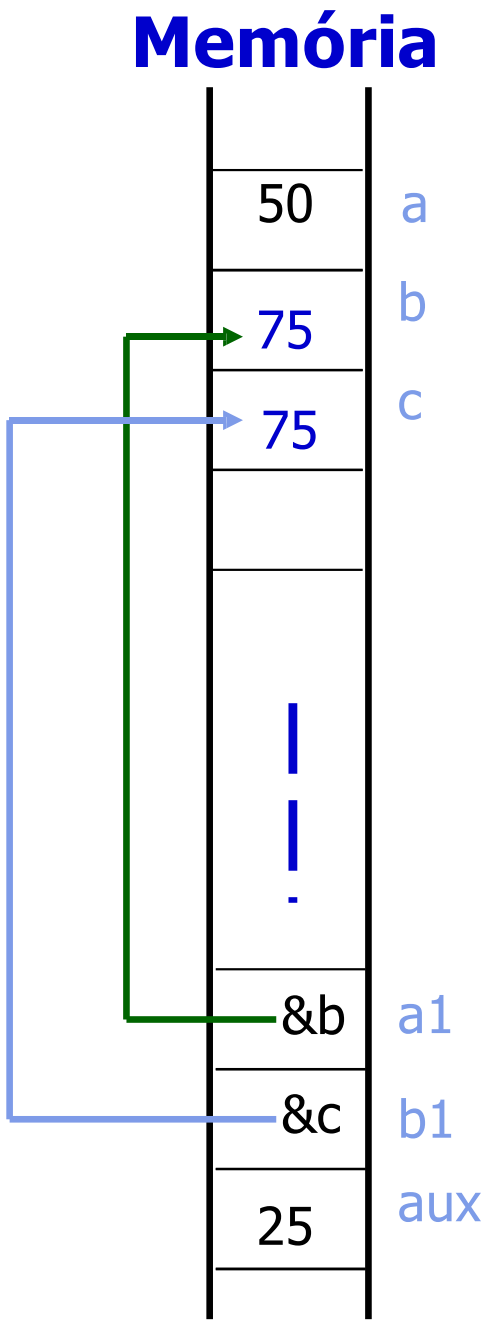
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



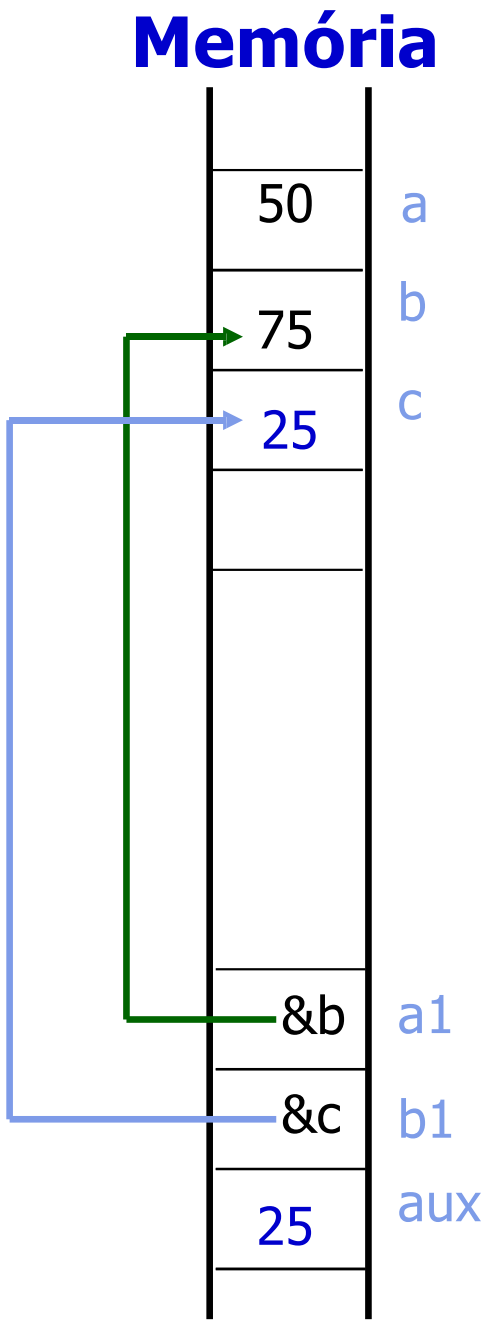
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



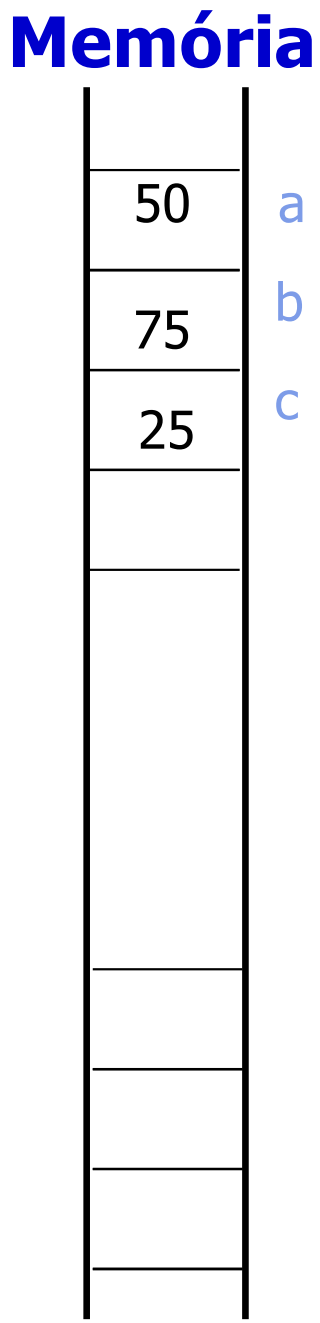
Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



Chamada por referência-exemplo

```
#include <stdio.h>
void Troca(int *a1, int *b1)
{
  int aux=*a1;
  *a1=*b1;
  *b1=aux;
}

int main()
{
 int a=25, b=50, c=75;
 Troca(&a,&b);
 Troca(&b,&c);
 printf("%d,%d,%d",a,b,c);
 return 0;
}
```



Exercícios *(continuação)*

5) O que será impresso no programa abaixo?

```
#include <stdio.h>

void calculo (int *p, int *q)
{
 *p = *p * 10;
 *q = *q + 10;
}

int main()
{
   int x = 2, y = 5;
   calculo(&x,&y);
   printf("%d - %d",x,y);
   return 0;
}
```

Exercícios

6) Qual a saída do programa abaixo:

```
#include <stdio.h>
int calculo (int p, int q)
{
 p = p * 10;
 q = q + 10;
 return(p + q);
}

int main()
{
   int x = 2, y = 5;
   printf("%d %d %d",x,y,calculo(x,y));
   return 0;
}
```

Exercícios

7) Qual a saída do programa abaixo:

```
#include <stdio.h>

int calculo (int p, int *q)
{
  p = p * 10;
  *q = *q + 10;
  return(p);
}

int main()
{
   int x = 2, y = 5, z;
   z = calculo(x,&y);
   printf("%d  %d  %d",x,y,z);
   return 0;
}
```

Exercícios

8) Qual a saída do programa abaixo:

```
#include <stdio.h>

int cal(int p, int *q, int *r)
{
  p = p * 10;
  *q = *q + 10;
  *r = *r - 10;
  return(p);
}

int main()
{
  int x = 2, y = 5, z = 3, r;
  r = cal(x,&y,&z);
  printf("%d  %d %d %d",x,y,z,r);
  return 0;
}
```

Vetores

- O vetor é uma estrutura:
  - Homogênea
  - Estática

- Todas as componentes são de um mesmo tipo e seu tamanho permanece o mesmo durante toda a execução do programa.

Vetores: Declaração

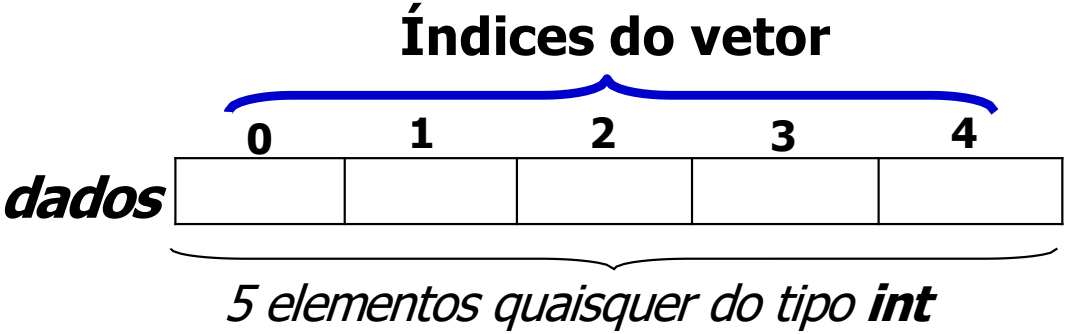
- A sintaxe em C para declaração de variável do tipo vetor é a seguinte

  tipo_primitivo identificador[qtde_elementos];

Exemplo:

// vetor com 5 (0 a 4) elementos do tipo int

int dados[5];



Vetores: Referência (Manipulação)

- Cada um dos elementos de um vetor é **referenciado individualmente por meio de um número inteiro entre colchetes após o nome do vetor**.

	0	1	2	3	4
valores	**3.6**	**2.7**	**4.2**	**7.9**	**1.2**

Exemplos:

- **X = valores[1]; //atribui a x o valor da posição 1 do vetor valores**
- **Y = valores[4]; //atribui a x o valor da posição 4 do vetor valores**
- **valores[0] = 3.2; // a posição zero do vetor valores recebe o valor 3.2**

Vetores: Inicialização

É possível fornecer valores a cada elemento de um vetor no momento da sua declaração.

Exemplo:

float valores[5] = {3.6, 2.7, 4.2, 7.9, 1.2};

	0	1	2	3	4
valores	3.6	2.7	4.2	7.9	1.2

float nros[5] = {0.0}; *// como fica o vetor????*

Vetores: Inicialização

Observações:

- Os vetores NÃO são automaticamente inicializados!!!

- Se houver menos valores do que o número de elementos do vetor , os elementos restantes são inicializados automaticamente com o valor zero.

```
int n[5] = {32, 64, 27};
```

- A seguinte declaração causa um erro de sintaxe:

```
int n[5] = {32, 64, 27, 18, 95, 14};
```

- O tamanho do vetores pode ser omitido:

```
int n[ ] = {1, 2, 3, 4, 5};
```

Vetores: Exemplo

- Leia um vetor de 10 posições (inteiros) e imprima-o na ordem invertida (da última para a primeira posição).

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int numeros[10],i;
   for (i=0; i<10;i++){
       printf("Digite valor %d: ",i);
       scanf("%d", &numeros[i]);
   }
   printf("Vetor na ordem invertida:\n");
   for (i=9; i>=0; i--)
      printf("%d\n",numeros[i]);
   return 0;

}
```

Vetores e Subrotinas

- Em C, vetores são passados sempre por referência.

- Ou seja, as modificações feitas na subrotina refletem nos dados do vetor passado como parâmetro pela função chamadora.

Exercícios

1)Desenvolva um programa que leia um vetor de números reais, um escalar e imprima o resultado da multiplicação do vetor pelo escalar.

2)Faça um procedimento que faça a leitura um vetor de 10 elementos inteiros e imprima somente os valores armazenados nos índices pares.

3)Faça um programa que leia um vetor com 15 valores reais. A seguir, encontre o menor elemento do vetor e a sua posição dentro do vetor, mostrando: “O menor elemento do vetor esta na posição XXXX e tem o valor YYYYY.”

4) Faça um programa que leia um vetor de 15 posições (reais) e depois um valor a ser procurado no vetor. Imprima se o valor foi ou não encontrado e a quantidade de vezes que o valor está presente no vetor.

Exercícios

5)Faça uma função que receba um vetor de números inteiros e um valor inteiro. A função deverá procurar este segundo valor neste vetor e retornar seu índice se for encontrado. Se o elemento não for encontrado, retornar -1.

6)Dada uma tabela com as notas de uma turma de 20 alunos, faça funções que retornem:

a) A média da turma.

b) a quantidade de alunos aprovados (>=60)

c) a quantidade de alunso reprovados.(< 60)

7)Faça um programa que leia um conjunto de 20 valores e armazene-os num vetor V. Particione-o em dois outros vetores, P e I, conforme os valores de V forem pares ou ímpares. No final, imprima os valores dos 3 vetores.

Exercícios

8) Faça um programa que leia um vetor G[13] que é o gabarito de um teste da loteria esportiva, contendo os valores 1 quando for coluna 1, 0 quando for coluna do meio e 2 quando for coluna 2.

Ler a seguir, para 5 apostadores, seu cartão de apostas (R[13]) e depois da leitura imprimir quantos acertos o apostador teve.

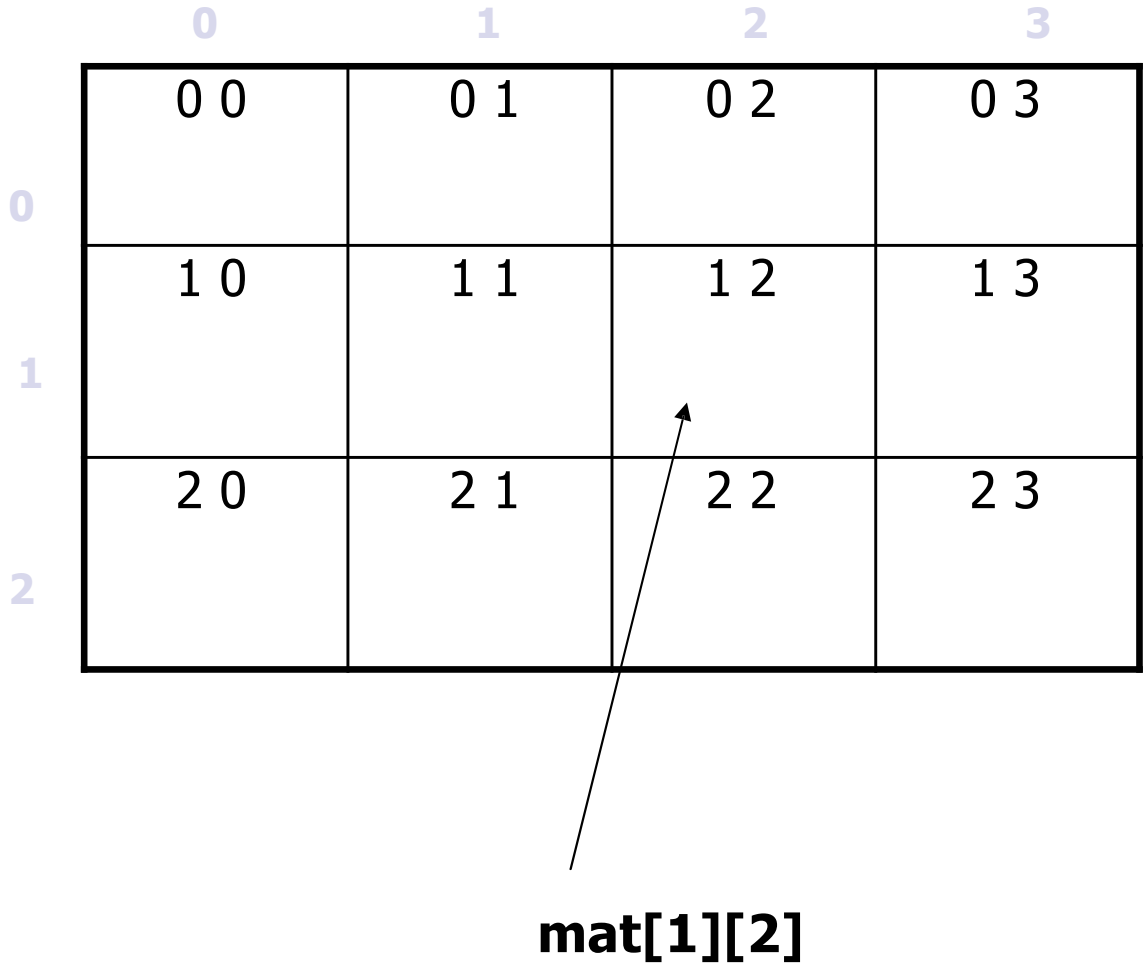
Faça o teste através de funções.

9) Com relação ao exercício anterior, calcule e mostre o percentual dos apostadores que fizeram de 10 a 13 pontos e o percentual dos apostadores que fizeram menos do que 10 pontos.

10) Faça um programa que leia um vetor de valores inteiros e imprima-o na ordem crescente. O vetor deve ter tamanho N (utilize a diretiva #define).

Declaração de matrizes

int mat[3][4];



Declaração de matrizes

- **Declaração de matrizes com mais de 2 dimensões**

float mat2[3][5][2];

int mat3[3][5][2][7];

etc…

- **Operações em matrizes**

mat2[2][4][1] = 8.6; mat3[0][0][0][1] = 5;

Matrizes: inicialização

Pode-se fornecer valores de cada elemento de uma matriz na declaração, da mesma forma que nos vetores

Exemplo:

```
float num[2][3] = {{3.6, 2.7,1.5},{5.0,4.1,2.3}};
```

Ou seja, fornece-se os valores linha a linha.

```
int val[5][2] = {{3,7}} // inicializou a primeira com os
        //valores 3 e 7. As demais receberão zero.
```

Matrizes: inicialização

Observações:

- As matrizes não são inicializadas automaticamente

- Se houver menos valores do que o número de elementos da matriz, os elementos restantes são inicializados automaticamente com o valor zero.

```
int num[5][3] = {{32, 64, 27}};
```

```
int n[4][4] = {{0}}; //todos elementos são nulos
```

- A seguinte declaração causa um erro de sintaxe:

```
int n[2][5] = {{32, 64, 27, 18, 95, 14},{12,15,43,17,67,31}};
```

Matrizes: exemplos

O programa abaixo inicializa os elementos de uma matriz **m** com os valores iguais a soma dos índices de cada elemento e imprime cada valor.

```
#include <stdio.h>

int main()
{
  int m[3][2], i,j;

  for (i=0; i < 3; i++)
    for(j=0; j < 2; j++)
    {
      m[i][j] = i+j;
      printf("i=%d j=%d elemento=%d\n",i,j,m[i][j]);
    }
  return 0;
}
```

```
i= 0 j= 0 elemento=0
i= 0 j= 1 elemento=1
i= 1 j= 0 elemento=1
i= 1 j= 1 elemento=2
i= 2 j= 0 elemento=2
i= 2 j= 1 elemento=3
```

Matrizes e subrotinas

- Matrizes serão passadas para subrotinas da mesma forma como os vetores são passados, a menos de um detalhe: **apenas a primeira dimensão pode ser omitida.**

  Ex.:

```
void imprimeMatriz(float m[][3])
```

ou

```
void imprimeMatriz(float m[3][3])
```

- **Havendo mais dimensões, a mesma regra deverá ser seguida.**

Exercícios

1)Faça um procedimento para calcular a multiplicação de uma matriz 3 x 4 por um escalar. Faça também um procedimento capaz de imprimir esta matriz. Ao final, desenvolva uma função principal onde será criada e lida uma matriz 3 x 4. Faça nesta função a chamada dos dois procedimentos criados anteriormente.

2)Faça um programa que leia uma matriz 7 x 5 e imprima a terceira coluna.

3)Faça um programa que leia uma matriz quadrada de dimensão 10, uma função que encontre o maior valor desta matriz e uma função que encontre o menor valor. Imprima os valores encontrados na função principal.

4)Faça um programa que leia uma matriz 6 x 3 e um procedimento que gere duas matrizes 3 x 3, a primeira com as 3 primeiras linhas e a outra com as restantes.

Exercícios

5)Faça um programa que leia uma matriz de caracteres 5 x 10 e uma função que conte quantas letras “a” aparecem na matriz. Você poderá fazer a leitura letra a letra ou considerar que cada linha da matriz é uma *string*.

6)Faça um programa que leia 3 vetores reais de dimensão 10 e um procedimento que gere uma matriz (10 x 3) onde cada coluna é dada por um destes vetores.

7)Crie uma matriz tridimensional onde as linhas indicam as notas de matemática, história e geografia em três provas de 10 alunos e crie uma função que verifique quantos alunos passaram, ou seja, os que tenham média aritmética > 60 nas 3 disciplinas.

Exercícios

8)Faça um programa para ler a quantidade de um total de 5 produtos que uma empresa tem em suas 7 lojas e imprimir em uma tabela:

a) o total de cada produto nestas lojas

b) a loja que tem menos produtos

9) Idem ao exercício 1, agora com o seguinte “Menu”:

Escolha uma opção de cálculo para matrizes:
1)Soma
2) Diferença
3) Transposta
4) Produto
5)Sair
Opção:__

Na opção (3) o usuário só precisa fornecer uma matriz, nas opções (1), (2) e (4) o usuário deve fornecer duas matrizes.

Observação, a matriz deve ser 3x3.

Exercícios

10) Implemente as subrotinas abaixo para que o programa funcione corretamente.

```
#include <stdio.h>
#define MAX 10
int  main()
{
   float a[MAX][MAX], b[MAX][MAX];
   int m, n;
   printf("\nDigite o no de linhas e de colunas das matrizes: ");
   scanf("%d,%d",&m,&n);
   printf("\nForneça os elementos da matriz a:\n");
   lematriz(a,m,n);   // leitura da matriz a
   printf("\nForneça os elementos da matriz b:\n");
   lematriz(b,m,n);  // leitura da matriz b
   copiazera(a,b,m,n); // a matriz a "recebe" b e b é zerada.
   printf("\nMatriz a:\n");
   imprimematriz(a,m,n);
   printf("\nMatriz b:\n");
   imprimematriz(b,m,n);
   return 0;
}
```

Estruturas de Dados Heterogêneas

- Até agora vimos as estruturas de dados homogêneas: **vetores**, **matrizes** e ***strings***.

- Nestas estruturas todos os elementos da estrutura são de tipos de dados primitivos: **inteiro, real, caractere**.

Estruturas de Dados Heterogêneas

- No entanto, em muitos casos, necessitamos armazenar um conjunto de informações relacionadas, formado por diversos tipos de dado primitivos.

- Exemplos:
  - Endereço
  - Fichas com dados pessoais de um cliente
  - Fichas com dados de um produto

Variáveis Compostas Heterogêneas

- Quando uma determinada estrutura de dados for composta por **diversos tipos diferentes,** primitivos ou não, temos um conjunto heterogêneo de dados.

- Essas variáveis são chamadas de variáveis compostas heterogêneas.

- Estas variáveis compostas são chamadas de estruturas (ou *structs* em C).

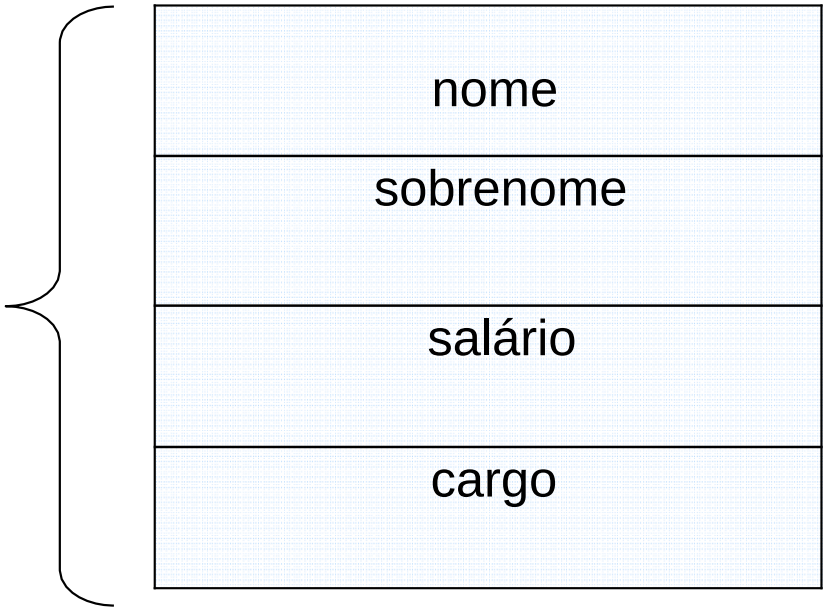
Definição de Estrutura

- Uma estrutura pode ser definida como uma coleção de uma ou mais variáveis relacionadas (campos), **onde cada variável pode ser de um tipo distinto**.

Exemplo:

empregado

(4 campos)



Estruturas em C

- Sintaxe para definir uma estrutura com **n** campos em C:

```
struct <nome_estrutura>
{
    tipo_1 IDENTIFICADOR_1;
    tipo_2 IDENTIFICADOR_2;
        ..................
    tipo_n IDENTIFICADOR_n;
};
```

Estruturas (declaração)

- Em C, pode-se criar uma variável de uma dada estrutura de duas formas:

```
struct <nome_estrutura> VAR_1, VAR_2,...,VAR_M;
```

ou

```
typedef <nome_estrutura> <novo_nome_estrutura>;

<novo_nome_estrutura> VAR_1, VAR_2,...,VAR_M;
```

Comando *typedef é usado para definir uma novo nome para a estrutura*

Estruturas: Declaração

- A sintaxe para declaração de uma variável deste tipo é a seguinte:

```
struct <nome_estrutura>
{
    tipo1 nome1;
    tipo2 nome2;  ...
};
struct <nome_estrutura> identif_1,identif_2,...;
```

Ou:

```
struct <nome_estrutura>
{
   tipo1 nome1;
   tipo2 nome2;...
} identificador1,identificador2,..;
```

Estrutura

- Veja um exemplo de estrutura que armazenaria a matricula e o nome de um funcionário.

Definição

```
struct funcionario
{
   char matricula[15];
   char nome[100];
};
```

Declaração

```
...
struct funcionario f1;
```

Estruturas: Manipulação

- Campos ou membros de uma estrutura podem ser usados da mesma forma como as variáveis.

- Campos são acessados usando o operador de acesso **ponto** (.) entre o **nome da estrutura** e o **nome do campo**.

Estruturas: Manipulação

- Para modificar um campo de uma estrutura, basta usarmos novamente o operador (.). No exemplo anterior, poderíamos ler o nome do funcionário *f1* da seguinte forma

```
struct funcionario f1;
gets(f1.nome);
```

Estrutura: Exemplo Completo

```
#include <stdio.h>

struct funcionario
{
    char matricula[15];
    char nome[100];
};

int main()
{
    struct funcionario f1;
    gets(f1.matricula);
    gets(f1.nome);

    puts("Informações armazenadas:\n");
    puts(f1.matricula);
    puts(f1.nome);
    return 0;
}
```

Estruturas: Outras formas de Declaração

```
struct endereco
{
    char rua [50];
     int numero;
     char bairro[20];
    char cidade[30];
    char sigla_estado[3];
    int  CEP;
}end;
```

Estruturas: Outras formas de Declaração

É possivel realizar as operações de definição, declaração e inicialização de estruturas em conjunto:

```
struct horario
{
      int horas;
      int minutos;
      int segundos;
} horario_do_nascimento = { 8, 45, 0 };
```

Typedef

- Permite a definição de um novo nome para um determinado tipo. A sintaxe é:

  typedef antigo_nome novo_nome;

- Exemplos:

  typedef int inteiro;

```
typedef struct scoord
{
    int x;
    int y;
}coord;
coord primeira,segunda;
```

Exemplo de uso do *typedef*

```
#include <stdio.h>
#include <stdlib.h>
typedef struct cad_aluno
{
   char  nome[40];
   float nota[4];
}aluno;
int main()
{
    aluno aluno1;
    int i;
    gets(aluno1.nome);
    for(i = 0; i <= 3; i++)
      scanf("%f", &aluno1.nota[i]);
    puts(aluno1.nome);
    for(i = 0; i <= 3; i++)
      printf("%f\n", aluno1.nota[i]);
    return 0;
}
```

Vetores de estruturas

```
typedef struct sproduto
{
    int  codigo;
    char descricao[120];
}produto;
```

- Após a definição da estrutura produto, pode-se definir um vetor de produtos como:

```
produto estoque[100];
```

Acesso aos elementos do vetor

```
typedef struct sproduto
{
    int  codigo;
    char descricao[120];
}produto;
```

- Utilizando o exemplo acima, pode-se modificar o campo código dos dois primeiros produtos da seguinte forma:

```
produto estoque[100];
...
estoque[0].codigo = 10;
estoque[1].codigo = 17;
```

Acesso aos elementos do vetor

```
typedef struct sproduto
{
    int  codigo;
    char descricao[120];
}produto;
```

- Ainda fazendo uso do exemplo acima, o fragmento de código para ler os 100 produtos ficaria assim:

```
produt  estoque[100];
o
...     i = 0; i < 100;
for(in  i++)
t  scanf("%d", &estoque[i].codigo);
{  gets(estoque[i].descricao);
}
```

Código completo do exemplo

```
#include <stdio.h>
#define TAM 100
typedef struct sproduto
{
    int  codigo;
    char descricao[120];
}produto;

int main()
{
   int i;
   produto estoque[TAM]; // Cria vetor de estruturas
   for( i = 0; i < TAM; i++)
   {
      printf("Informe o codigo do produto %d: ", i);
      scanf("%d%*c", &estoque[i].codigo); // %*c descarta o '\n'
      printf("Informe a descricao do produto %d: ", i);
      gets(estoque[i].descricao);
   }

   printf("Listagem dos produtos armazenados:\n");
   for( i = 0; i < TAM; i++)
      printf("%3d - %s\n", estoque[i].codigo,estoque[i].descricao);
   return 0;
}
```

Inicialização de um vetor de estruturas

```
typedef struct sproduto
{
    int  codigo;
    char descricao[120];
}produto;
```

- Pode-se inicializar um vetor de estruturas no momento de sua criação como o código abaixo:

```
produto estoque[3] =
   {235, "Teclado USB",
    245, "Monitor LCD",
    515, "Memoria DDR"};
```

Inicializando um vetor de estruturas

```
typedef struct sproduto
{
    int  codigo;
    char descricao[120];
}produto;
```

- Se inicializarmos menos elementos do que os alocados em memória, o restante do vetor ficará "zerado" (se numérico) ou vazio (se *strings*).

```
produto estoque[6] =
   {235, "Teclado USB",
    245, "Monitor LCD",
    515, "Memoria DDR"};
```

- No exemplo acima, os índices 3, 4 e 5 do vetor terão código 0 e descrição vazia.

Funções e vetores de estruturas

- Vetores de estruturas podem ser passados como argumentos de funções como qualquer outro vetor.

```
...
produto estoque[100];
lerProdutos(estoque);
...
```

```
void lerProdutos( produto estoque[100]
)
{
  for(int i = 0; i < 100; i++)
  {
     scanf("%d", &estoque[i].codigo);
     gets(estoque[i].descricao);
  }
}
```

Estruturas e funções – Exemplo 1/3

```
#include <stdio.h>
#define TAM 4

typedef struct sproduto
{
    int  codigo;
    char descricao[120];
}produto;

.................

int main()
{
   produto estoque[TAM];  // Cria vetor de estruturas
   LerProdutos(estoque);     // Chama função de leitura
   ImprimeProdutos(estoque); // Chama função de impressão
   return 0;
}
```

Estruturas e funções – Exemplo 2/3

```
..........
void LerProdutos( produto estoque[TAM] )
{
 int i;
 printf("Listagem dos produtos armazenados:\n");
 for( i = 0; i < TAM; i++)
 {
  printf("Informe o codigo do produto %d: ", i);
  scanf("%d%*c", &estoque[i].codigo); // %*c descarta o '\n'
  printf("Informe a descricao do produto %d: ", i);
  gets(estoque[i].descricao);
 }
}
...............
```

Estruturas e funções – Exemplo 3/3

```
..........

void ImprimeProdutos( produto estoque[TAM] )
{
 int i;
 for( i = 0; i < TAM; i++)
  printf("%3d - %s\n",estoque[i].codigo, estoque[i].descricao);
}


.........
```

Estruturas e funções - Completo

```
#include <stdio.h>
#define TAM 4

typedef struct sproduto
{
    int  codigo;
    char descricao[120];
}produto;

void LerProdutos( produto estoque[TAM] )
{
   int i;
   printf("Listagem dos produtos armazenados:\n");
   for( i = 0; i < TAM; i++)
   {
      printf("Informe o codigo do produto %d: ", i);
      scanf("%d%*c", &estoque[i].codigo); // %*c descarta o '\n'
      printf("Informe a descricao do produto %d: ", i);
      gets(estoque[i].descricao);
   }
}
```

Estruturas e funções - Completo

- Continuação...

```
void ImprimeProdutos( produto estoque[TAM] )
{
   int i;
   for( i = 0; i < TAM; i++)
      printf("%3d - %s\n", estoque[i].codigo,estoque[i].descricao);
}

int main()
{
   int i;
   produto estoque[TAM];       // Cria vetor de estruturas
   LerProdutos(estoque);       // Chama função de leitura
   ImprimeProdutos(estoque);   // Chama função de impressão
   return 0;
}
```

Exercícios

1) Faça um programa que leia as informações de *N* alunos (sendo *N* definido com a diretiva *define*). As informações que deverão ser lidas de cada aluno são: matrícula, nome e média final. Ao final, informe os nomes dos alunos que foram aprovados (>= 60).

2) Faça um programa para fazer o cadastro dos veículos de uma empresa. Poderão ser cadastrados no máximo 100 veículos e as informações a serem armazenadas são as seguintes: nome do condutor, placa do veículo e cor do veículo. Armazene também o turno que os veículos são armazenados como sendo *m* (manhã), *t* (tarde), *n* (noite) e *i* (dia inteiro). Informe ao final a lista completa dos veículos cadastro com todas as informações que forem cadastradas. Utilize procedimentos para fazer a leitura e a escrita dos dados.

3) Faça um programa que gerencie o estoque de uma empresa. Faça a leitura dos produtos que contenham os seguintes campos: nome do produto, quantidade em estoque e preço unitário. Ao final, informe as informações de todos os produtos. Informe também os produtos que tenham o maior e o menor valor em estoque. Calcule este valor multiplicando a quantidade de cada produto pelo seu valor unitário. Utilize funções para realizar operações de leitura e escrita e faça um menu que possibilite ler todos os produtos, ler um produto com um índice específico e imprimir as informações citadas anteriormente.

Exercícios

4) Elabore um programa que auxilie no controle de uma fazenda de gado que possua um total de 100 cabeças de gado. O programa deverá conter uma estrutura que comporte:

- código: código da cabeça de gado;
- leite: número de litros de leite produzido por semana;
- alimento: quantidade de alimento ingerida por semana - em quilos;
- mês de nascimento;
- ano de nascimento;
- abate: ‘N”(não) ou ‘S’ (sim).

Exercícios

4) continuação...

O seu programa deverá **conter um menu** com as seguintes funcionalidades:

(a) Ler a base de dados (código, leite, alimento, nascimento) informados pelo usuário e armazenar em um vetor de estruturas.

(b) Preencher o campo *abate*, considerando que a cabeça de gado irá para o abate caso:

- tenha mais de 5 anos, ou;
- produza menos de 40 litros de leite por semana, ou;
- produza entre 50 e 70 litros de leite por semana e ingira mais de 50 quilos de alimento por semana.

(c) Imprimir a quantidade total de leite produzida por semana na fazenda.

(d) Imprimir a quantidade total de alimento consumido por semana na fazenda.

(e) Imprimir a quantidade total de leite que vai ser produzido por semana na fazenda, após o abate

(f) Imprimir a quantidade total de alimento que vai ser consumido por semana na fazenda, após o abate

(g) Imprimir número de cabeças de gado que irão para o abate.

(h) Inclua uma opção para sair do menu.

Exercícios

1) Defina uma estrutura para representar as informações de um cartão de crédito.

2) Defina uma estrutura para representar o peso a altura de uma pessoa.

3) Considerando a estrutura do exercício (2) e a inicialização de variáveis abaixo:

```
pessoa joao, maria;
joao.altura ← 1.90; joao.peso ← 98;
maria.altura ← 1.50; maria.peso ← 55;
```

Escreva uma instrução que atribua 1.78 à altura de `joao`;

Escreva uma instrução que atribua 75 ao peso de `maria`.

Escreva um conjunto de instruções para imprimir a média das alturas e a média dos pesos de `joao` e `maria`.

Exercícios

4) Faça um programa (função principal) para leitura, via teclado, dos dados de um aluno. Os dados a serem guardados na estrutura aluno são os seguintes: nome, curso, idade. Ao final, imprima estas informações na tela.

5) Considere a mesma estrutura definida anteriormente. Acrescente à estrutura um vetor com as notas das três provas feitas pelo aluno, calcule a sua média e diga se ele foi aprovado ou não (media >= 60). Para o cálculo da média e leitura das notas, utilize o comando `para` (que "varrerá" o vetor da estrutura).

6) Crie uma estrutura chamada `ponto` contendo apenas as coordenadas `x` e `y` (inteiros) do ponto. Declare 2 pontos, leia as coordenadas `x` e `y` de cada um e calcule a distância entre eles. Apresente no final a distância entre os dois pontos.

Exercícios

7) Faça um programa que permita a entrada de registros com nome, cidade e telefone de 5 pessoas. Imprima ao final os dados armazenados.

8) Faça um programa que receba três nomes de no máximo 100 caracteres cada e as idades das respectivas pessoas em um vetor de estruturas. Após o recebimento, listar os três nomes e idades armazenados neste vetor.

9) Faça um programa que armazene as informações de 11 jogadores de um time de futebol. Cada jogador é identificado pelo número da sua camisa, pelo seu peso, altura e pela inicial do seu nome. O programa deverá ler todas as informações e imprimir ao final a lista completa de jogadores com todas as informações lidas, a inicial do jogador mais baixo e o número do mais pesado.

Exercícios

10) Faça um programa que permita a entrada de registros com CPF, nome, idade, cidade e telefone de no máximo 100 pessoas. Faça um menu com as seguintes opções:

```
Menu:
1 – Ler as informações de uma pessoa
2 – Imprima por idade
3 – Imprima por inicial
4 – Imprima todos os registros cadastrados
```

O programa deverá ter as seguintes características:

- No primeiro item, peça inicialmente o índice do vetor que deseja alterar.
- No segundo item, peça a idade mínima e máxima e imprima as pessoas que estão neste intervalo.
- Faça funções para realizar as operações de cada um dos itens do menu.
- Inicialize o CPF de todos os 100 registros no início do programa com -1. Utilize essa informação no item 4 para definir se um registro foi ou não lido naquela posição específica do vetor. CPF com valor -1 simbolizam posições não lidas do vetor.

Tipos Abstratos de Dados (TAD)

- Modelo matemático, acompanhado das operações definidas sobre o modelo.
  - Exemplo: o conjunto dos inteiros acompanhado das operações de adição, subtração e multiplicação.
- TADs são utilizados como base para o projeto de algoritmos.
- A implementação do algoritmo em uma linguagem de programação exige a representação do TAD em termos dos tipos de dados e dos operadores suportados.
- A representação do modelo matemático por trás do tipo abstrato de dados é realizada mediante uma estrutura de dados.
- Podemos considerar TADs como generalizações de tipos primitivos e procedimentos como generalizações de operações primitivas.
- O TAD encapsula tipos de dados. A definição do tipo e todas as operações ficam localizadas numa seção do programa.

Implementação de TADs (1)

- Considere uma uma lista de inteiros. Poderíamos definir TAD Lista, com as seguintes operações:
  1. faça a lista vazia;
  2. obtenha o primeiro elemento da lista; se a lista estiver vazia, então retorne nulo;
  3. insira um elemento na lista.
- Há várias opções de estruturas de dados que permitem uma implementação eficiente para listas (por ex., o tipo estruturado arranjo).

Implementação de TADs (2)

- Cada operação do tipo abstrato de dados é implementada como um procedimento na linguagem de programação escolhida.
- Qualquer alteração na implementação do TAD fica restrita à parte encapsulada, sem causar impactos em outras partes do código.
- Cada conjunto diferente de operações define um TAD diferente, mesmo que atuem sob um mesmo modelo matemático.
- A escolha adequada de uma implementação depende fortemente das operações a serem realizadas sobre o modelo.

Conteúdo do Capítulo

1. Listas Lineares
   1. Implementação de Listas por meio de Arranjos
   2. Implementação de Listas por meio de Apontadores
2. Pilhas
   1. Implementação de Pilhas por meio de Arranjos
   2. Implementação de Pilhas por meio de Apontadores
3. Filas
   1. Implementação de Filas por meio de Arranjos
   2. Implementação de Filas por meio de Apontadores

Listas Lineares

- Uma das formas mais simples de interligar os elementos de um conjunto.
- Estrutura em que as operações inserir, retirar e localizar são definidas.
- Podem crescer ou diminuir de tamanho durante a execução de um programa, de acordo com a demanda.
- Itens podem ser acessados, inseridos ou retirados de uma lista.
- Duas listas podem ser concatenadas para formar uma lista única, ou uma pode ser partida em duas ou mais listas.
- Adequadas quando não é possível prever a demanda por memória, permitindo a manipulação de quantidades imprevisíveis de dados, de formato também imprevisível.
- São úteis em aplicações tais como manipulação simbólica, gerência de memória, simulação e compiladores.

Definição de Listas Lineares

- Seqüência de zero ou mais itens $x_1, x_2, \cdots, x_n$, na qual x_i é de um determinado tipo e n representa o tamanho da lista linear.
- Sua principal propriedade estrutural envolve as posições relativas dos itens em uma dimensão.
  - Assumindo $n \geq 1$, x_1 é o primeiro item da lista e x_n é o último item da lista.
  - x_i precede x_{i+1} para $i = 1, 2, \cdots, n-1$
  - x_i sucede x_{i-1} para $i = 2, 3, \cdots, n$
  - o elemento x_i é dito estar na i-ésima posição da lista.

TAD Listas Lineares

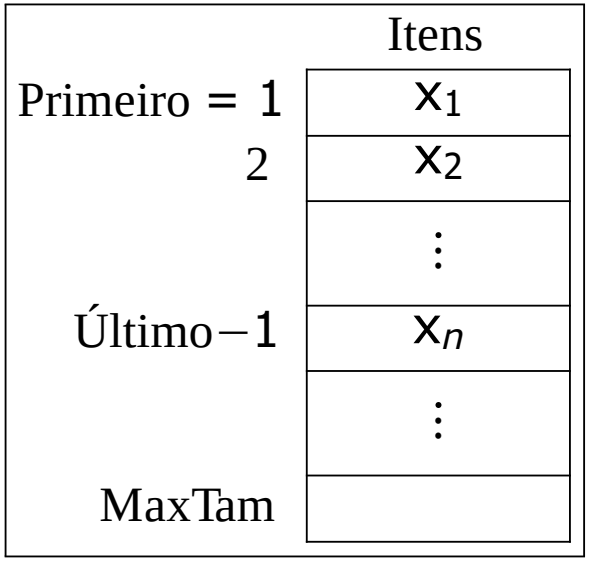
- O conjunto de operações a ser definido depende de cada aplicação.
- Um conjunto de operações necessário a uma maioria de aplicações é:
  1. Criar uma lista linear vazia.
  2. Inserir um novo item imediatamente após o *i*-ésimo item.
  3. Retirar o *i*-ésimo item.
  4. Localizar o *i*-ésimo item para examinar e/ou alterar o conteúdo de seus componentes.
  5. Combinar duas ou mais listas lineares em uma lista única.
  6. Partir uma lista linear em duas ou mais listas.
  7. Fazer uma cópia da lista linear.
  8. Ordenar os itens da lista em ordem ascendente ou descendente, de acordo com alguns de seus componentes.
  9. Pesquisar a ocorrência de um item com um valor particular em algum componente.

Implementações de Listas Lineares

- Várias estruturas de dados podem ser usadas para representar listas lineares, cada uma com vantagens e desvantagens particulares.
- As duas representações mais utilizadas são as implementações por meio de arranjos e de apontadores.
- Exemplo de Conjunto de Operações:
  1. FLVazia(Lista). Faz a lista ficar vazia.
  2. Insere(x, Lista). Insere x após o último item da lista.
  3. Retira(p, Lista, x). Retorna o item x que está na posição p da lista, retirando-o da lista e deslocando os itens a partir da posição p+1 para as posições anteriores.
  4. Vazia(Lista). Esta função retorna *true* se lista vazia; senão retorna *false*.
  5. Imprime(Lista). Imprime os itens da lista na ordem de ocorrência.

Implementação de Listas por meio de Arranjos

- Os itens da lista são armazenados em posições contíguas de memória.
- A lista pode ser percorrida em qualquer direção.
- A inserção de um novo item pode ser realizada após o último item com custo constante.



- A inserção de um novo item no meio da lista requer um deslocamento de todos os itens localizados após o ponto de inserção.
- Retirar um item do início da lista requer um deslocamento de itens para preencher o espaço deixado vazio.

Estrutura da Lista Usando Arranjo

```
#define INICIOARRANJO 1
#define MAXTAM 1000

typedef int TipoApontador;
typedef int TipoChave;
typedef struct {
  TipoChave Chave;
  /*--- outros componentes --- */
} TipoItem;
typedef struct {
  TipoItem Item[MAXTAM];
  TipoApontador Primeiro, Ultimo;
} TipoLista;
```

- Os itens são armazenados em um **array** de tamanho suficiente para armazenar a lista.
- O campo Último aponta para a posição seguinte a do último elemento da lista.
- O *i*-ésimo item da lista está armazenado na *i*-ésima posição do **array**, $1 \leq i <$Último.
- A constante MaxTam define o tamanho máximo permitido para a lista.

Operações sobre Lista Usando Arranjo

```
void FLVazia(TipoLista *Lista)
{ Lista->Primeiro = INICIOARRANJO; Lista->Ultimo = Lista->Primeiro; }

int Vazia(TipoLista Lista)
{ return (Lista.Primeiro == Lista.Ultimo); }  /* Vazia */

void Insere(TipoItem x, TipoLista *Lista)
{ if (Lista -> Ultimo > MAXTAM)
    printf("Lista esta cheia\n");
  else { Lista -> Item[Lista -> Ultimo - 1] = x;
         Lista -> Ultimo++;
       }
}  /* Insere */
```

Operações sobre Lista Usando Arranjo

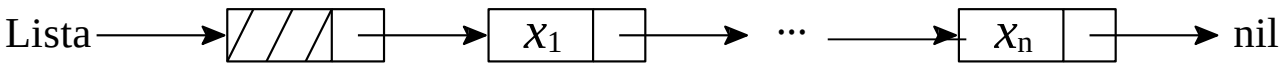
```
void Retira(TipoApontador p, TipoLista *Lista, TipoItem *Item)
{ int Aux;
  if (Vazia(*Lista) || p >= Lista -> Ultimo)
  { printf("Erro: Posicao nao existe\n");
    return;
  }
  *Item = Lista -> Item[p - 1];
  Lista -> Ultimo--;
  for (Aux = p; Aux < Lista -> Ultimo; Aux++)
    Lista -> Item[Aux - 1] = Lista -> Item[Aux];
}  /* Retira */
void Imprime(TipoLista Lista)
{ int Aux;
  for (Aux = Lista.Primeiro - 1; Aux <= (Lista.Ultimo - 2); Aux++)
    printf("%d\n", Lista.Item[Aux].Chave);
}  /* Imprime */
```

Lista Usando Arranjo - Vantagens e Desvantagens

- Vantagem: economia de memória (os apontadores são implícitos nesta estrutura).
- Desvantagens:
  - custo para inserir ou retirar itens da lista, que pode causar um deslocamento de todos os itens, no pior caso;
  - em aplicações em que não existe previsão sobre o crescimento da lista, a utilização de arranjos em linguagens como o Pascal pode ser problemática porque nesse caso o tamanho máximo da lista tem de ser definido em tempo de compilação.

Implementação de Listas por meio de Apontadores

- Cada item é encadeado com o seguinte mediante uma variável do tipo Apontador.
- Permite utilizar posições não contíguas de memória.
- É possível inserir e retirar elementos sem necessidade de deslocar os itens seguintes da lista.
- Há uma **célula cabeça** para simplificar as operações sobre a lista.



Estrutura da Lista Usando Apontadores

```
typedef int TipoChave;
typedef struct {
  TipoChave Chave;
  /* outros componentes */
} TipoItem;
typedef struct TipoCelula *TipoApontador;
typedef struct TipoCelula {
  TipoItem Item;
  TipoApontador Prox;
} TipoCelula;

typedef struct {
  TipoApontador Primeiro, Ultimo;
} TipoLista;
```

- A lista é constituída de células.
- Cada célula contém um item da lista e um apontador para a célula seguinte.
- O registro TipoLista contém um apontador para a célula cabeça e um apontador para a última célula da lista.

Operações sobre Lista Usando Apontadores

```
void FLVazia(TipoLista *Lista)
{ Lista->Primeiro = (TipoApontador) malloc(sizeof(TipoCelula));
  Lista->Ultimo = Lista->Primeiro; Lista->Primeiro->Prox = NULL;
}

int Vazia(TipoLista Lista)
{ return (Lista.Primeiro == Lista.Ultimo); }

void Insere(TipoItem x, TipoLista *Lista)
{ Lista->Ultimo->Prox = (TipoApontador) malloc(sizeof(TipoCelula));
  Lista->Ultimo = Lista->Ultimo->Prox; Lista->Ultimo->Item = x;
  Lista->Ultimo->Prox = NULL;
}
```

Operações sobre Lista Usando Apontadores

```
void Retira(TipoApontador p, TipoLista *Lista, TipoItem *Item)
{ /*--O item a ser retirado e o seguinte ao apontado por p--*/
  TipoApontador q;
  if (Vazia(*Lista) || p == NULL || p->Prox == NULL)
  { printf(" Erro: Lista vazia ou posicao nao existe\n");
    return;
  }
  q = p->Prox; *Item = q->Item;   p->Prox = q->Prox;
  if (p->Prox == NULL) Lista->Ultimo = p;
  free(q);
}
void Imprime(TipoLista Lista)
{ TipoApontador Aux;
  Aux = Lista.Primeiro->Prox;
  while (Aux != NULL)
    { printf("%d\n", Aux->Item.Chave); Aux = Aux->Prox; }
}
```

Listas Usando Apontadores - Vantagens e Desvantagens

- Vantagens:
  - Permite inserir ou retirar itens do meio da lista a um custo constante (importante quando a lista tem de ser mantida em ordem).
  - Bom para aplicações em que não existe previsão sobre o crescimento da lista (o tamanho máximo da lista não precisa ser definido *a priori*).
- Desvantagem: utilização de memória extra para armazenar os apontadores.

Exemplo de Uso Listas - Vestibular

- Num vestibular, cada candidato tem direito a três opções para tentar uma vaga em um dos sete cursos oferecidos.
- Para cada candidato é lido um registro:
  - Chave: número de inscrição do candidato.
  - NotaFinal: média das notas do candidato.
  - Opção: vetor contendo as três opções de curso do candidato.

```
Chave     : 1..999;
NotaFinal: 0..10;
Opcao     : array[1..3] of 1..7;
```

- Problema: distribuir os candidatos entre os cursos, segundo a nota final e as opções apresentadas por candidato.
- Em caso de empate, os candidatos serão atendidos na ordem de inscrição para os exames.

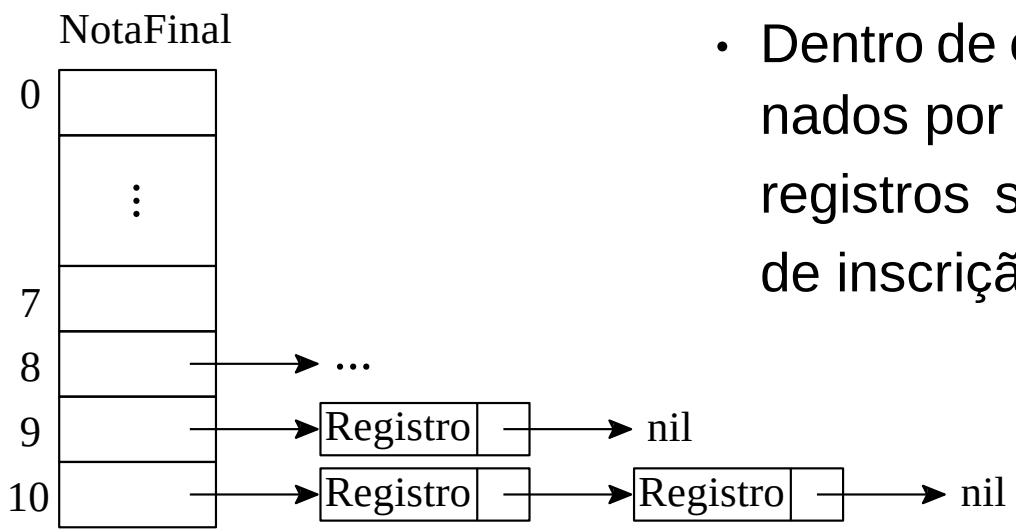
Vestibular - Possível Solução

- Ordenar registros por NotaFinal, respeitando a ordem de inscrição.
- Percorrer registros com mesma NotaFinal, começando pelo conjunto de NotaFinal 10, depois NotaFinal 9, e assim por diante.
  - Para um conjunto de mesma NotaFinal encaixar cada registro em um dos cursos, na primeira opção em que houver vaga (se houver).
- Primeiro refinamento:

```
int Nota; ordena os registros pelo campo NotaFinal;
for (Nota = 10; Nota >= 0; Nota--)
  { while houver registro com mesma nota
      { if  existe vaga em um dos cursos de opção do candidato
        {insere registro no conjunto de aprovados}
        else insere registro no conjunto de reprovados;
      }
  }
imprime aprovados por curso; imprime reprovados;
```

Vestibular - Classificação dos Alunos

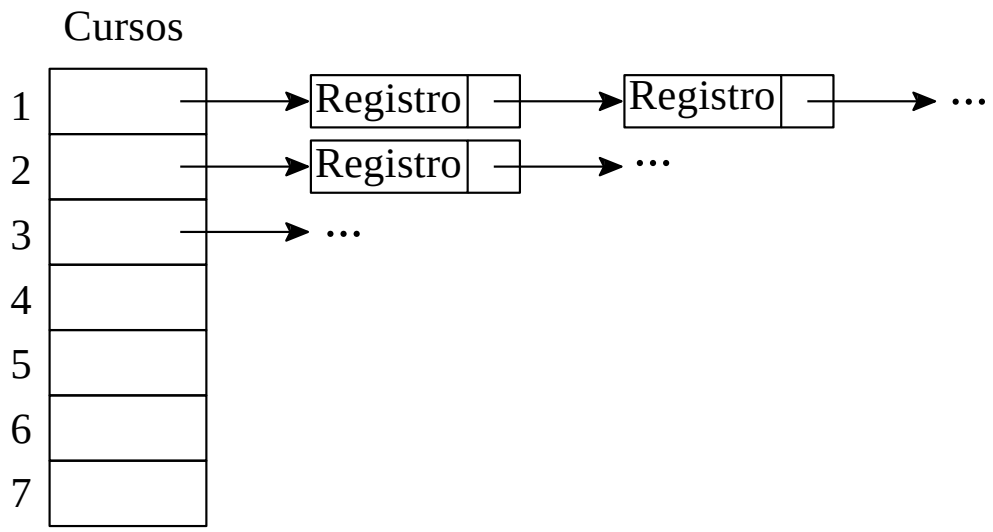
- Uma boa maneira de representar um conjunto de registros é com o uso de listas.
- Os registros são armazenados em listas para cada nota.
- Após a leitura do último registro os candidatos estão automaticamente ordenados por NotaFinal.



- Dentro de cada lista, os registros estão ordenados por ordem de inscrição, desde que os registros sejam lidos e inseridos na ordem de inscrição de cada candidato.

Vestibular - Classificação dos Alunos

- As listas de registros são percorridas, iniciando-se pela de NotaFinal 10, seguida pela de NotaFinal 9, e assim sucessivamente.
- Cada registro é retirado e colocado em uma das listas da abaixo, na primeira das três opções em que houver vaga.



- Se não houver vaga, o registro é colocado em uma lista de reprovados.
- Ao final a estrutura acima conterá a relação de candidatos aprovados em cada curso.

Vestibular - Segundo Refinamento

```
int Nota; TipoChave Chave; lê número de vagas para cada curso;
inicializa listas de classificação, de aprovados e de reprovados;
lê registro;
while ( Chave ! = 0 )
   { insere registro nas listas de classificação, conforme nota final; lê registro; }
for ( Nota= 10; Nota>= 0; Nota−−)
   { while ( houver próximo registro com mesma NotaFinal )
      { retira registro da lista;
         if existe vaga em um dos cursos de opção do candidato
         { insere registro na lista de aprovados;
            decrementa o número de vagas para aquele curso;
         }
         else { insere registro na lista de reprovados; }
         obtém próximo registro;
      }
   }
   imprime aprovados por curso; imprime reprovados;
```

Vestibular - Estrutura Final da Lista

```
#define NOPCOES 3
#define NCURSOS 7
typedef short TipoChave;
typedef struct TipoItem {
  TipoChave Chave;
  int NotaFinal;
  int Opcao[NOPCOES];
} TipoItem;
typedef struct TipoCelula*TipoApontador;
typedef struct TipoCelula {
  TipoItem Item;
  TipoApontador Prox;
} TipoCelula;
typedef struct TipoLista {
  TipoApontador Primeiro, Ultimo;
} TipoLista;
```

Vestibular - Refinamento Final (1)

```
#define NOPCOES        3
#define NCURSOS        7
#define FALSE          0
#define TRUE           1
/*--- Entram aqui os tipos do Slide 21 ---*/
TipoItem Registro;
TipoLista Classificacao[11];
TipoLista Aprovados[NCURSOS];
TipoLista Reprovados;
long Vagas[NCURSOS];  short Passou;  long i, Nota;
/*--- Entram aqui os operadores sobre listas dos Slides 13 e 14 ---*/
void LeRegistro(TipoItem *Registro)
{ /*--- os valores lidos devem estar separados por brancos ---*/
  long i;
  scanf("%ld %d", &Registro->Chave, &Registro->NotaFinal);
  for (i = 0; i < NOPCOES; i++) scanf("%d", &Registro->Opcao[i]);
}
```

Vestibular - Refinamento Final (2)

```
{ /*--Programa principal--*/
  for (i = 1; i <= NCURSOS; i++) scanf("%d", &Vagas[i-1]);
  scanf("%*[^\n]");
  getchar();
  for (i = 0; i <= 10; i++) FLVazia(&Classificacao[i]);
  for (i = 1; i <= NCURSOS; i++) FLVazia(&Aprovados[i-1]);
  FLVazia(&Reprovados);
  LeRegistro(&Registro);
  while (Registro.Chave != 0)
    {
      Insere(Registro, &Classificacao[Registro.NotaFinal]);
      LeRegistro(&Registro);
    }
```

Vestibular - Refinamento Final (3)

```
  for (Nota = 10; Nota >= 0; Nota--)
    { while (!Vazia(Classificacao[Nota]))
        { Retira(Classificacao[Nota].Primeiro, &Classificacao[Nota], &Registro);
          i = 1;  Passou = FALSE;
        while (i <= NOPCOES && !Passou)
          { if (Vagas[Registro.Opcao[i-1] - 1] > 0)
              { Insere(Registro, &Aprovados[Registro.Opcao[i-1] - 1]);
                Vagas[Registro.Opcao[i-1] - 1]--;  Passou = TRUE;
              }
              i++;
          }
        if (!Passou) Insere(Registro, &Reprovados);
        }
    }
  for (i = 1; i <= NCURSOS; i++)
   { printf("Relacao dos aprovados no Curso%d\n",i); Imprime(Aprovados[i-1]); }
  printf("Relacao dos reprovados\n");  Imprime(Reprovados);
  return 0;
}
```

Vestibular - Refinamento Final

- Observe que o programa é completamente independente da implementação do tipo abstrato de dados Lista.
- O exemplo mostra a importância de utilizar **tipos abstratos de dados** para escrever programas, em vez de utilizar detalhes particulares de implementação.
- Altera-se a implementação rapidamente. Não é necessário procurar as referências diretas às estruturas de dados por todo o código.
- Esse aspecto é importante em programas de grande porte.

Pilha

- É uma lista linear em que todas as inserções, retiradas e, geralmente, todos os acessos são feitos em apenas um extremo da lista.
- Os itens são colocados um sobre o outro. O item inserido mais recentemente está no topo e o inserido menos recentemente no fundo.
- O modelo intuitivo é o de um monte de pratos em uma prateleira, sendo conveniente retirar ou adicionar pratos na parte superior.
- Esta imagem está freqüentemente associada com a teoria de autômato, na qual o topo de uma pilha é considerado como o receptáculo de uma cabeça de leitura/gravação que pode empilhar e desempilhar itens da pilha.

Propriedade e Aplicações das Pilhas

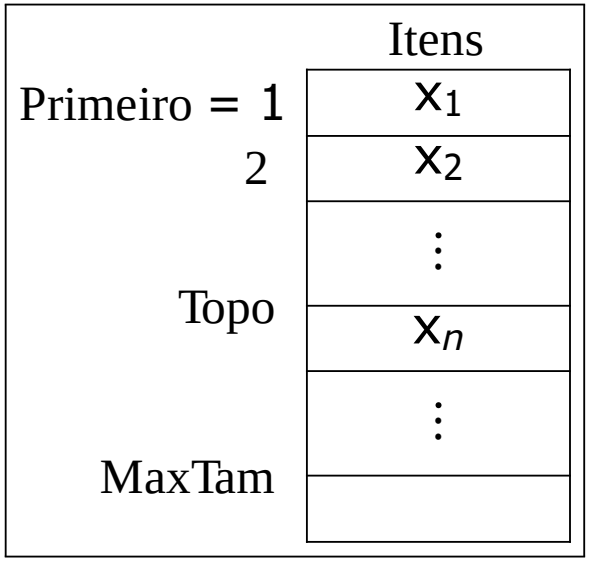
- Propriedade: o último item inserido é o primeiro item que pode ser retirado da lista. São chamadas listas **lifo** (“last-in, first-out”).
- Existe uma ordem linear para pilhas, do “mais recente para o menos recente”.
- É ideal para estruturas aninhadas de profundidade imprevisível.
- Uma pilha contém uma seqüência de obrigações adiadas. A ordem de remoção garante que as estruturas mais internas serão processadas antes das mais externas.
- Aplicações em estruturas aninhadas:
  - Quando é necessário caminhar em um conjunto de dados e guardar uma lista de coisas a fazer posteriormente.
  - O controle de seqüências de chamadas de subprogramas.
  - A sintaxe de expressões aritméticas.
- As pilhas ocorrem em estruturas de natureza recursiva (como árvores). Elas são utilizadas para implementar a **recursividade**.

TAD Pilhas

- Conjunto de operações:
  1. FPVazia(Pilha). Faz a pilha ficar vazia.
  2. Vazia(Pilha). Retorna *true* se a pilha está vazia; caso contrário, retorna *false*.
  3. Empilha(x, Pilha). Insere o item x no topo da pilha.
  4. Desempilha(Pilha, x). Retorna o item x no topo da pilha, retirando-o da pilha.
  5. Tamanho(Pilha). Esta função retorna o número de itens da pilha.
- Existem várias opções de estruturas de dados que podem ser usadas para representar pilhas.
- As duas representações mais utilizadas são as implementações por meio de *arranjos* e de *apontadores*.

Implementação de Pilhas por meio de Arranjos

- Os itens da pilha são armazenados em posições contíguas de memória.
- Como as inserções e as retiradas ocorrem no topo da pilha, um cursor chamado Topo é utilizado para controlar a posição do item no topo da pilha.



Estrutura da Pilha Usando Arranjo

- Os itens são armazenados em um **array** do tamanho da pilha.
- O outro campo do mesmo registro contém um apontador para o item no topo da pilha.
- A constante MaxTam define o tamanho máximo permitido para a pilha.

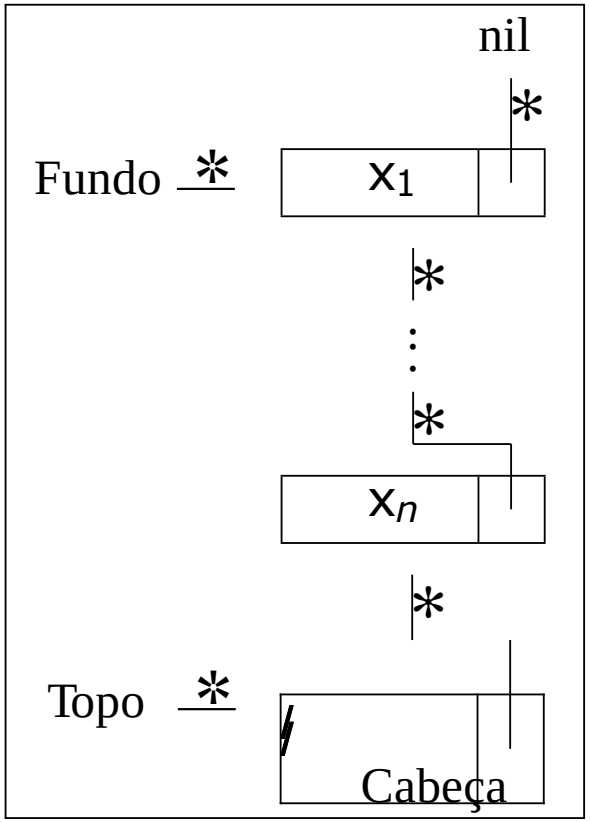
```
#define MAXTAM 1000
typedef int TipoApontador;
typedef int TipoChave;
typedef struct {
  TipoChave Chave;
  /*--- outros componentes --- */
} TipoItem;
typedef struct {
  TipoItem Item[MAXTAM];
  TipoApontador Topo;
} TipoPilha;
```

Operações sobre Pilhas Usando Arranjos

```
void FPVazia(TipoPilha *Pilha)
{ Pilha->Topo = 0; }

int Vazia(TipoPilha Pilha)
{ return (Pilha.Topo == 0); }

void Empilha(TipoItem x, TipoPilha *Pilha)
{ if (Pilha->Topo == MaxTam) printf("Erro: pilha esta cheia\n");
  else { Pilha->Topo++; Pilha->Item[Pilha->Topo - 1] = x; }
}
void Desempilha(TipoPilha *Pilha, TipoItem *Item)
{ if (Vazia(*Pilha)) printf("Erro: pilha esta vazia\n");
  else { *Item = Pilha->Item[Pilha->Topo - 1]; Pilha->Topo --; }
}
int Tamanho(TipoPilha Pilha)
{ return (Pilha.Topo); }
```

Implementação de Pilhas por meio de Apontadores

- Há uma célula cabeça no topo para facilitar a implementação das operações empilha e desempilha quando a pilha está vazia.
- Para desempilhar o item x_n basta desligar a célula cabeça da lista e a célula que contém x_n passa a ser a célula cabeça.
- Para empilhar um novo item, basta fazer a operação contrária, criando uma nova célula cabeça e colocando o item na antiga.



Estrutura da Pilha Usando Apontadores

- O campo Tamanho evita a contagem do número de itens na função Tamanho.
- Cada célula de uma pilha contém um item da pilha e um apontador para outra célula.
- O registro TipoPilha contém um apontador para o topo da pilha (célula cabeça) e um apontador para o fundo da pilha.

```
typedef int TipoChave;
typedef struct {
  int Chave;
  /* outros componentes */
} TipoItem;
typedef struct TipoCelula *TipoApontador;
typedef struct TipoCelula {
  TipoItem Item;
  TipoApontador Prox;
} TipoCelula;
typedef struct {
  TipoApontador Fundo, Topo;
  int Tamanho;
} TipoPilha;
```

Operações sobre Pilhas Usando Apontadores

```
void FPVazia(TipoPilha *Pilha)
{ Pilha->Topo = (TipoApontador) malloc(sizeof(TipoCelula));
  Pilha->Fundo = Pilha->Topo;
  Pilha->Topo->Prox = NULL;
  Pilha->Tamanho = 0;
}
int Vazia(TipoPilha Pilha)
{ return (Pilha.Topo == Pilha.Fundo); }

void Empilha(TipoItem x, TipoPilha *Pilha)
{ TipoApontador Aux;
  Aux = (TipoApontador) malloc(sizeof(TipoCelula));
  Pilha->Topo->Item = x;
  Aux->Prox = Pilha->Topo;
  Pilha->Topo = Aux;
  Pilha->Tamanho++;
}
```

Operações sobre Pilhas Usando Apontadores

```
void Desempilha(TipoPilha *Pilha, TipoItem *Item)
{ TipoApontador q;
  if (Vazia(*Pilha)) { printf("Erro: lista vazia\n"); return; }
  q = Pilha->Topo;
  Pilha->Topo = q->Prox;
  *Item = q->Prox->Item;
  free(q);  Pilha->Tamanho--;
}

int Tamanho(TipoPilha Pilha)
{ return (Pilha.Tamanho); }
```

Exemplo de Uso Pilhas - Editor de Textos (ET)

- "#": cancelar caractere anterior na linha sendo editada. Ex.: UEM##FMB#G → UFMG.
- "\": cancela todos os caracteres anteriores na linha sendo editada.
- "*": salta a linha. Imprime os caracteres que pertencem à linha sendo editada, iniciando uma nova linha de impressão a partir do caractere imediatamente seguinte ao caractere salta-linha. Ex: DCC*UFMG.* → DCC UFMG.
- Vamos escrever um Editor de Texto (ET) que aceite os três comandos descritos acima.
- O ET deverá ler um caractere de cada vez do texto de entrada e produzir a impressão linha a linha, cada linha contendo no máximo 70 caracteres de impressão.
- O ET deverá utilizar o **tipo abstrato de dados** Pilha definido anteriormente, implementado por meio de arranjo.

Sugestão de Texto para Testar o ET

Este et# um teste para o ET, o extraterrestre em

PASCAL.*Acabamos de testar a capacidade de o ET saltar de linha,

utilizando seus poderes extras (cuidado, pois agora vamos estourar

a capacidade máxima da linha de impressão, que é de 70

caracteres.)*O k#cut#rso dh#e Estruturas de Dados et# h#um

cuu#rsh#o #x# x?*!#?!#+.* Como et# bom

n#nt#ao### r#ess#tt#ar mb#aa#triz#cull#ado nn#x#ele!\ Sera

que este funciona\\\? O sinal? não### deve ficar! ~

ET - Implementação

- Este programa utiliza um tipo abstrato de dados sem conhecer detalhes de sua implementação.
- A implementação do TAD Pilha que utiliza arranjo pode ser substituída pela que utiliza apontadores sem causar impacto no programa.

```
#define MAXTAM 70
#define CANCELACARATER '#'
#define CANCELALINHA '\\'
#define SALTALINHA '*'
#define MARCAEOF '~'
typedef char TipoChave;
/* Entram aqui os tipos da transparência 30 */
var Pilha: TipoPilha;
    x      : TipoItem;
/* Entram aqui os operadores da transparência 31 */
/* Entra aqui o procedimento Imprime (transp. 40) */
```

ET - Implementação

```
int main(int argc, char *argv[])
{ TipoPilha Pilha; TipoItem x;
  FPVazia(&Pilha);  x.Chave = getchar();
  if (x.Chave == '\n') x.Chave = ' ';
  while (x.Chave != MARCAEOF)
  { if (x.Chave == CANCELACARATER)
    { if (!Vazia(Pilha)) Desempilha(&Pilha, &x);}
    else if (x.Chave == CANCELALINHA) FPVazia(&Pilha);
         else if (x.Chave == SALTALINHA) Imprime(&Pilha);
              else { if (Tamanho(Pilha) == MAXTAM) Imprime(&Pilha);
                     Empilha(x, &Pilha);
                   }
    x.Chave = getchar(); if (x.Chave == '\n') x.Chave = ' ';
  }
  if (!Vazia(Pilha)) Imprime(&Pilha); return 0;
}
```

ET - Implementação (Procedimento Imprime)

```
void Imprime(TipoPilha *Pilha)
{ TipoPilha Pilhaux;
  TipoItem x;
  FPVazia(&Pilhaux);
  while (!Vazia(*Pilha))
    { Desempilha(Pilha, &x); Empilha(x, &Pilhaux);
    }
  while (!Vazia(Pilhaux))
    { Desempilha(&Pilhaux, &x); putchar(x.Chave);
    }
  putchar('\n');
}
```

Fila

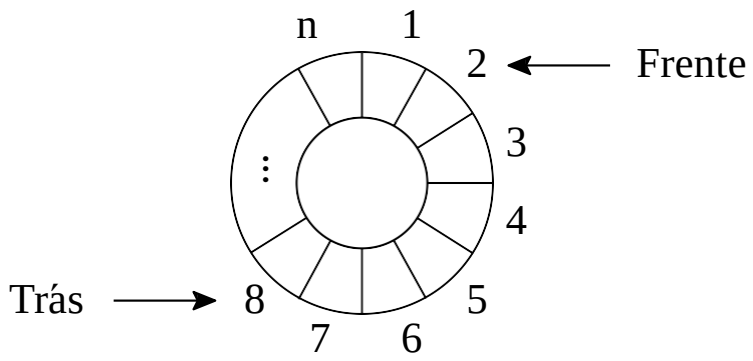
- É uma lista linear em que todas as inserções são realizadas em um extremo da lista, e todas as retiradas e, geralmente, os acessos são realizados no outro extremo da lista.
- O modelo intuitivo de uma fila é o de uma fila de espera em que as pessoas no início da fila são servidas primeiro e as pessoas que chegam entram no fim da fila.
- São chamadas listas **fifo** (“first-in”, “first-out”).
- Existe uma ordem linear para filas que é a “ordem de chegada”.
- São utilizadas quando desejamos processar itens de acordo com a ordem “primeiro-que-chega, primeiro-atendido”.
- Sistemas operacionais utilizam filas para regular a ordem na qual tarefas devem receber processamento e recursos devem ser alocados a processos.

TAD Filas

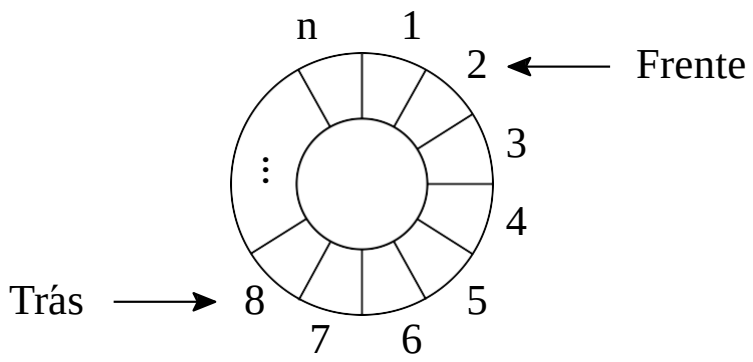
- Conjunto de operações:
  1. FFVazia(Fila). Faz a fila ficar vazia.
  2. Enfileira(x, Fila). Insere o item x no final da fila.
  3. Desenfileira(Fila, x). Retorna o item x no início da fila, retirando-o da fila.
  4. Vazia(Fila). Esta função retorna *true* se a fila está vazia; senão retorna *false*.

Implementação de Filas por meio de Arranjos

- Os itens são armazenados em posições contíguas de memória.
- A operação Enfileira faz a parte de trás da fila expandir-se.
- A operação Desenfileira faz a parte da frente da fila contrair-se.
- A fila tende a caminhar pela memória do computador, ocupando espaço na parte de trás e descartando espaço na parte da frente.
- Com poucas inserções e retiradas, a fila vai ao encontro do limite do espaço da memória alocado para ela.
- Solução: imaginar o **array** como um círculo. A primeira posição segue a última.



Implementação de Filas por meio de Arranjos



- A fila se encontra em posições contíguas de memória, em alguma posição do círculo, delimitada pelos apontadores Frente e Trás.
- Para enfileirar, basta mover o apontador Trás uma posição no sentido horário.
- Para desenfileirar, basta mover o apontador Frente uma posição no sentido horário.

Estrutura da Fila Usando Arranjo

- O tamanho do **array** circular é definido pela constante MaxTam.
- Os outros campos do registro TipoPilha contêm apontadores para a parte da frente e de trás da fila.

```
#define MAXTAM 1000
typedef int TipoApontador;
typedef int TipoChave;
typedef struct {
  TipoChave Chave;
  /* outros componentes */
} TipoItem;
typedef struct {
  TipoItem Item[MAXTAM];
  TipoApontador Frente, Tras;
} TipoFila;
```

Operações sobre Filas Usando Arranjos

- Nos casos de fila cheia e fila vazia, os apontadores Frente e Trás apontam para a mesma posição do círculo.
- Uma saída para distinguir as duas situações é deixar uma posição vazia no **array**.
- Nesse caso, a fila está cheia quando Trás+1 for igual a Frente.

```
void FFVazia(TipoFila *Fila)
{ Fila->Frente = 1;
  Fila->Tras = Fila->Frente;
}

int Vazia(TipoFila Fila)
{ return (Fila.Frente == Fila.Tras); }
```

Operações sobre Filas Usando Arranjos

- A implementação utiliza aritmética modular nos procedimentos Enfileira e Desenfileira (função **mod** do Pascal).

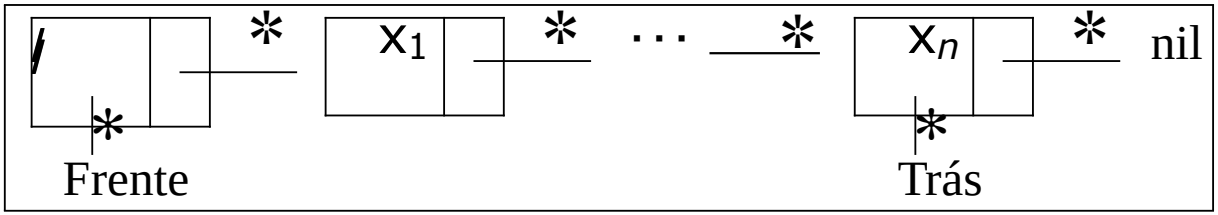
```
void Enfileira(TipoItem x, TipoFila *Fila)
{ if (Fila->Tras % MAXTAM + 1 == Fila->Frente)
  printf(" Erro  fila est a cheia\n");
  else { Fila->Item[Fila->Tras - 1] = x;
         Fila->Tras = Fila->Tras % MAXTAM + 1;
       }
}
void Desenfileira(TipoFila *Fila, TipoItem *Item)
{ if (Vazia(*Fila))
  printf("Erro fila esta vazia\n");
  else { *Item = Fila->Item[Fila->Frente - 1];
         Fila->Frente = Fila->Frente % MAXTAM + 1;
       }
}
```

Operações sobre Filas Usando Arranjos

```
void Imprime(TipoFila *Fila)
{

}
```

Implementação de Filas por meio de Apontadores

- Há uma célula cabeça é para facilitar a implementação das operações Enfileira e Desenfileira quando a fila está vazia.
- Quando a fila está vazia, os apontadores Frente e Trás apontam para a célula cabeça.
- Para enfileirar um novo item, basta criar uma célula nova, ligá-la após a célula que contém x_n e colocar nela o novo item.
- Para desenfileirar o item x_1, basta desligar a célula cabeça da lista e a célula que contém x_1 passa a ser a célula cabeça.



Estrutura da Fila Usando Apontadores

- A fila é implementada por meio de células.
- Cada célula contém um item da fila e um apontador para outra célula.
- O registro TipoFila contém um apontador para a frente da fila (célula cabeça) e um apontador para a parte de trás da fila.

```
typedef struct TipoCelula *TipoApontador;
typedef int TipoChave;
typedef struct TipoItem {
  TipoChave Chave;
  /* outros componentes */
} TipoItem;
typedef struct TipoCelula {
  TipoItem Item;
  TipoApontador Prox;
} TipoCelula;
typedef struct TipoFila {
  TipoApontador Frente, Tras;
} TipoFila;
```

Operações sobre Filas Usando Apontadores

```
void FFVazia(TipoFila *Fila)
{ Fila->Frente = (TipoApontador) malloc(sizeof(TipoCelula));
  Fila->Tras = Fila->Frente;
  Fila->Frente->Prox = NULL;
}

int Vazia(TipoFila Fila)
{ return (Fila.Frente == Fila.Tras); }

void Enfileira(TipoItem x, TipoFila *Fila)
{ Fila->Tras->Prox = (TipoApontador) malloc(sizeof(TipoCelula));
  Fila->Tras = Fila->Tras->Prox;
  Fila->Tras->Item = x;
  Fila->Tras->Prox = NULL;
}
```

Operações sobre Filas Usando Apontadores

```
void Desenfileira(TipoFila *Fila, TipoItem *Item)
{ TipoApontador q;
  if (Vazia(*Fila)) { printf("Erro fila esta vazia\n"); return; }
  q = Fila->Frente;
  Fila->Frente = Fila->Frente->Prox;
  *Item = Fila->Frente->Item;
  free(q);
}
```

Ordenação*

Última alteração: 31 de Agosto de 2010

*Transparências elaboradas por Charles Ornelas Almeida, Israel Guerra e Nivio Ziviani

Conteúdo do Capítulo

1. Ordenação Interna
   1. Seleção
   2. Inserção
   3. Shellsort
   4. Quicksort
   5. Heapsort
      - Filas de Prioridades
      - Heaps
   6. Ordenação Parcial
      - Seleção Parcial
      - Inserção Parcial
      - Heapsort Parcial
      - Quicksort Parcial
   7. Ordenação em Tempo Linear
      - Ordenação por Contagem
      - Radixsort para Inteiros
      - Radixsort para Cadeias de Caracteres
2. Ordenação Externa
   1. Intercalação Balanceada de Vários Caminhos
   2. Implementação por meio de Seleção por Substituição

   4.2.3 Considerações Práticas

   4.2.4 Intercalação Polifásica

   4.2.5 Quicksort Externo

Introdução - Conceitos Básicos

- Ordenar: processo de rearranjar um conjunto de objetos em uma ordem ascendente ou descendente.
- A ordenação visa facilitar a recuperação posterior de itens do conjunto ordenado.
  - Dificuldade de se utilizar um catálogo telefônico se os nomes das pessoas não estivessem listados em ordem alfabética.
- Notação utilizada nos algoritmos:
  - Os algoritmos trabalham sobre os registros de um arquivo.
  - Cada registro possui uma **chave** utilizada para controlar a ordenação.
  - Podem existir outros componentes em um registro.

Introdução - Conceitos Básicos

- Estrutura de um registro:

```
typedef long TipoChave;
typedef struct TipoItem {
  TipoChave Chave;
  /* outros componentes */
} TipoItem;
```

- Qualquer tipo de chave sobre o qual exista uma regra de ordenação bem-definida pode ser utilizado.
- Um método de ordenação é **estável** se a ordem relativa dos itens com chaves iguais não se altera durante a ordenação.
- Alguns dos métodos de ordenação mais eficientes não são estáveis.
- A estabilidade pode ser forçada quando o método é não-estável.
- Sedgewick (1988) sugere agregar um pequeno índice a cada chave antes de ordenar, ou então aumentar a chave de alguma outra forma.

Introdução - Conceitos Básicos

- Classificação dos métodos de ordenação:
  - Interna: arquivo a ser ordenado cabe todo na memória principal.
  - Externa: arquivo a ser ordenado não cabe na memória principal.
- Diferenças entre os métodos:
  - Em um método de ordenação interna, qualquer registro pode ser imediatamente acessado.
  - Em um método de ordenação externa, os registros são acessados seqüencialmente ou em grandes blocos.
- A maioria dos métodos de ordenação é baseada em **comparações** das chaves.
- Existem métodos de ordenação que utilizam o princípio da **distribuição**.

Introdução - Conceitos Básicos

- Exemplo de ordenação por distribuição: considere o problema de ordenar um baralho com 52 cartas na ordem:

$$A < 2 < 3 < \cdots < 10 < J < Q < K$$

e

$$\clubsuit < \diamondsuit < \heartsuit < \spadesuit.$$

- Algoritmo:
  1. Distribuir as cartas em treze montes: ases, dois, três, . . ., reis.
  2. Colete os montes na ordem especificada.
  3. Distribua novamente as cartas em quatro montes: paus, ouros, copas e espadas.
  4. Colete os montes na ordem especificada.

Introdução - Conceitos Básicos

- Métodos como o ilustrado são também conhecidos como **ordenação digital**, **radixsort** ou **bucketsort**.
- O método não utiliza comparação entre chaves.
- Uma das dificuldades de implementar este método está relacionada com o problema de lidar com cada monte.
- Se para cada monte nós reservarmos uma área, então a demanda por memória extra pode tornar-se proibitiva.
- O custo para ordenar um arquivo com n elementos é da ordem de $O(n)$.

Ordenação Interna

- Na escolha de um algoritmo de ordenação interna deve ser considerado o tempo gasto pela ordenação.
- Sendo n o número registros no arquivo, as medidas de complexidade relevantes são:
  - Número de comparações $C(n)$ entre chaves.
  - Número de movimentações $M(n)$ de itens do arquivo.
- O uso econômico da memória disponível é um requisito primordial na ordenação interna.
- Métodos de ordenação ***in situ*** são os preferidos.
- Métodos que utilizam listas encadeadas não são muito utilizados.
- Métodos que fazem cópias dos itens a serem ordenados possuem menor importância.

Ordenação Interna

- Classificação dos métodos de ordenação interna:
  - Métodos simples:
    - Adequados para pequenos arquivos.
    - Requerem $O(n^2)$ comparações.
    - Produzem programas pequenos.
  - Métodos eficientes:
    - Adequados para arquivos maiores.
    - Requerem $O(n \log n)$ comparações.
    - Usam menos comparações.
    - As comparações são mais complexas nos detalhes.
    - Métodos simples são mais eficientes para pequenos arquivos.

Ordenação Interna

- Tipos de dados e variáveis utilizados nos algoritmos de ordenação interna:

```
typedef int TipoIndice;
typedef TipoItem TipoVetor[MAXTAM + 1];
/* MAXTAM + 1 por causa da sentinela em Insercao */
TipoVetor A;
```

- O índice do vetor vai de 0 até $MaxTam$, devido às chaves **sentinelas**.
- O vetor a ser ordenado contém chaves nas posições de 1 até n.

Ordenação por Seleção (1)

- Um dos algoritmos mais simples de ordenação.
- Algoritmo:
  - Selecione o menor item do vetor.
  - Troque-o com o item da primeira posição do vetor.
  - Repita essas duas operações com os $n - 1$ itens restantes, depois com os $n - 2$ itens, até que reste apenas um elemento.

Ordenação por Seleção (2)

- O método é ilustrado abaixo:

	1	2	3	4	5	6
Chaves iniciais:	*O*	*R*	*D*	*E*	*N*	*A*
i = 1	***A***	*R*	*D*	*E*	*N*	***O***
i = 2	*A*	***D***	***R***	*E*	*N*	*O*
i = 3	*A*	*D*	***E***	***R***	*N*	*O*
i = 4	*A*	*D*	*E*	***N***	***R***	*O*
i = 5	*A*	*D*	*E*	*N*	***O***	***R***

- As chaves em negrito sofreram uma troca entre si.

Ordenação por Seleção

```
void Selecao(TipoItem *A, TipoIndice n)
{ TipoIndice i, j, Min;
  TipoItem x;
  for (i = 1; i <= n - 1; i++)
    { Min = i;
      for (j = i + 1; j <= n; j++)
        if (A[j].Chave < A[Min].Chave) Min = j;
      x = A[Min]; A[Min] = A[i]; A[i] = x;
    }
}
```

- Comparações entre chaves e movimentações de registros:

$$C(n) = \frac{n^2}{2} - \frac{n}{2}$$

$$M(n) = 3(n - 1)$$

- A atribuição $Min := j$ é executada em média $n \log n$ vezes, Knuth (1973).

Ordenação por Seleção

Vantagens:

- Custo linar para o número de movimentos de registros.
- É o algoritmo a ser utilizado para arquivos com registros muito grandes.
- É muito interessante para arquivos pequenos.

Desvantagens:

- O fato de o arquivo já estar ordenado não ajuda em nada, pois o custo continua quadrático.
- O algoritmo não é **estável**.

Ordenação por Inserção

- Método preferido dos jogadores de **cartas**.
- Algoritmo:
  - Em cada passo a partir de i=2 faça:
    - Selecione o *i*-ésimo item da seqüência fonte.
    - Coloque-o no lugar apropriado na seqüência destino de acordo com o critério de ordenação.

Ordenação por Inserção

- O método é ilustrado abaixo:

	1	2	3	4	5	6
Chaves iniciais:	***O***	*R*	*D*	*E*	*N*	*A*
i = 2	***O***	***R***	*D*	*E*	*N*	*A*
i = 3	***D***	***O***	***R***	*E*	*N*	*A*
i = 4	***D***	***E***	***O***	***R***	*N*	*A*
i = 5	***D***	***E***	***N***	***O***	***R***	*A*
i = 6	***A***	***D***	***E***	***N***	***O***	***R***

- As chaves em negrito representam a seqüência destino.

Ordenação por Inserção

```
void Insercao(TipoItem *A, TipoIndice n)
{ TipoIndice i, j;
  TipoItem x;
  for (i = 2; i <= n; i++)
    { x = A[i];  j = i - 1;
      A[0] = x;  /* sentinela */
      while (x.Chave < A[j].Chave)
        { A[j+1] = A[j];  j--;
        }
      A[j+1] = x;
    }
}
```

Ordenação por Inserção

Considerações sobre o algoritmo:

- O processo de ordenação pode ser terminado pelas condições:
  - Um item com chave menor que o item em consideração é encontrado.
  - O final da seqüência destino é atingido à esquerda.
- Solução:
  - Utilizar um registro **sentinela** na posição zero do vetor.

Ordenação por Inserção

- Seja $C(n)$ a função que conta o número de comparações.
- No anel mais interno, na i-ésima iteração, o valor de C_i é:

$$\begin{aligned}
&\text{Melhor caso} && : C_i(n) = 1 \\
&\text{Pior caso} && : C_i(n) = i \\
&\text{Caso médio} && : C_i(n) = \frac{1}{i}(1 + 2 + \cdots + i) = \frac{i+1}{2}
\end{aligned}$$

- Assumindo que todas as permutações de n são igualmente prováveis no caso médio, temos:

$$\begin{aligned}
&\text{Melhor caso} && : C(n) = (1 + 1 + \cdots + 1) = n - 1 \\
&\text{Pior caso} && : C(n) = (2 + 3 + \cdots + n) = \frac{n^2}{2} + \frac{n}{2} - 1 \\
&\text{Caso médio} && : C(n) = \frac{1}{2}(3 + 4 + \cdots + n + 1) = \frac{n^2}{4} + \frac{3n}{4} - 1
\end{aligned}$$

Ordenação por Inserção

- Seja $M(n)$ a função que conta o número de movimentações de registros.

- O número de movimentações na i-ésima iteração é:

$$M_i(n) = C_i(n) - 1 + 3 = C_i(n) + 2$$

- Logo, o número de movimentos é:

$$\begin{aligned}
&\text{Melhor caso} && : M(n) = (3 + 3 + \cdots + 3) = 3(n-1) \\
&\text{Pior caso} && : M(n) = (4 + 5 + \cdots + n + 2) = \frac{n^2}{2} + \frac{5n}{2} - 3 \\
&\text{Caso médio} && : M(n) = \frac{1}{2}(5 + 6 + \cdots + n + 3) = \frac{3n^2}{4} + \frac{11n}{4} - \frac{3}{4}
\end{aligned}$$

Ordenação por Inserção

- O número mínimo de comparações e movimentos ocorre quando os itens estão originalmente em ordem.
- O número máximo ocorre quando os itens estão originalmente na ordem reversa.
- É o método a ser utilizado quando o arquivo está “quase” ordenado.
- É um bom método quando se deseja adicionar uns poucos itens a um arquivo ordenado, pois o custo é linear.
- O algoritmo de ordenação por inserção é **estável**.

Shellsort

- Proposto por Shell em 1959.
- É uma extensão do algoritmo de ordenação por inserção.
- Problema com o algoritmo de ordenação por inserção:
  - Troca itens adjacentes para determinar o ponto de inserção.
  - São efetuadas $n - 1$ comparações e movimentações quando o menor item está na posição mais à direita no vetor.
- O método de Shell contorna este problema permitindo trocas de registros distantes um do outro.

Shellsort

- Os itens separados de *h* posições são rearranjados.
- Todo *h*-ésimo item leva a uma seqüência ordenada.
- Tal seqüência é dita estar *h*-ordenada.
- Exemplo de utilização:

	1	2	3	4	5	6
Chaves iniciais:	*O*	*R*	*D*	*E*	*N*	*A*
h = 4	*N*	*A*	*D*	*E*	*O*	*R*
h = 2	*D*	*A*	*N*	*E*	*O*	*R*
h = 1	*A*	*D*	*E*	*N*	*O*	*R*

- Quando $h = 1$ Shellsort corresponde ao algoritmo de inserção.

Shellsort

- Como escolher o valor de *h*:
  - Seqüência para *h*:

$$h(s) = 3h(s-1) + 1, \quad para\ s > 1$$

$$h(s) = 1, \quad para\ s = 1.$$

  - Knuth (1973, p. 95) mostrou experimentalmente que esta seqüência é difícil de ser batida por mais de 20% em eficiência.
  - A seqüência para *h* corresponde a 1, 4, 13, 40, 121, 364, 1.093, 3.280, . . .

Shellsort

```
void Shellsort(TipoItem *A, TipoIndice n)
{ int i, j;  int h = 1;
  TipoItem x;
  do h = h * 3 + 1; while (h < n);
  do
    { h /= 3;
      for (i = h + 1; i <= n; i++)
        { x = A[i];  j = i;
          while (A[j - h].Chave > x.Chave)
            { A[j] = A[j - h]; j -= h;
              if (j <= h) goto L999;
            }
            L999: A[j] = x;
        }
    } while (h != 1);
}
```

Shellsort

```
void shellSort(int vet[], int size) {
    int i , j , value;
    int gap = 1;
    while(gap < size) {
        gap = 3*gap+1;
    }
    while ( gap > 1) {
        gap /= 3;
        for(i = gap; i < size; i++) {
            value = vet[i];
            j = i - gap;
            while (j >= 0 && value < vet[j]) {
                vet [j + gap] = vet[j];
                j -= gap;
            }
            vet [j + gap] = value;
        }
    }
}
```

Shellsort

- A implementação do Shellsort não utiliza registros **sentinelas**.
- Seriam necessários *h* registros sentinelas, uma para cada *h*-ordenação.

Shellsort: Análise

- A razão da eficiência do algoritmo ainda não é conhecida.
- Ninguém ainda foi capaz de analisar o algoritmo.
- A sua análise contém alguns problemas matemáticos muito difíceis.
- A começar pela própria seqüência de incrementos.
- O que se sabe é que cada incremento não deve ser múltiplo do anterior.
- Conjecturas referente ao número de comparações para a seqüência de Knuth:

$$\text{Conjetura 1} \quad : C(n) = O(n^{1,25})$$

$$\text{Conjetura 2} \quad : C(n) = O(n(\ln n)^2)$$

Shellsort

- Vantagens:
  - Shellsort é uma ótima opção para arquivos de tamanho moderado.
  - Sua implementação é simples e requer uma quantidade de código pequena.
- Desvantagens:
  - O tempo de execução do algoritmo é sensível à ordem inicial do arquivo.
  - O método não é **estável**,

Quicksort

- Proposto por Hoare em 1960 e publiccado em 1962.
- É o algoritmo de ordenação interna mais rápido que se conhece para uma ampla variedade de situações.
- Provavelmente é o mais utilizado.
- A idéia básica é dividir o problema de ordenar um conjunto com *n* itens em dois problemas menores.
- Os problemas menores são ordenados independentemente.
- Os resultados são combinados para produzir a solução final.

Quicksort

- A parte mais delicada do método é o processo de partição.
- O vetor $A[\mathrm{Esq}..\mathrm{Dir}]$ é rearranjado por meio da escolha arbitrária de um **pivô** x.
- O vetor A é particionado em duas partes:
  - A parte esquerda com chaves menores ou iguais a x.
  - A parte direita com chaves maiores ou iguais a x.

Quicksort

- Algoritmo para o particionamento:
  1. Escolha arbitrariamente um **pivô** x.
  2. Percorra o vetor a partir da esquerda até que $A[i] \geq x$.
  3. Percorra o vetor a partir da direita até que $A[j] \leq x$.
  4. Troque $A[i]$ com $A[j]$.
  5. Continue este processo até os apontadores i e j se cruzarem.
- Ao final, o vetor $A[\mathrm{Esq}..\mathrm{Dir}]$ está particionado de tal forma que:
  - Os itens em $A[\mathrm{Esq}], A[\mathrm{Esq}+1], \ldots, A[j]$ são menores ou iguais a x.
  - Os itens em $A[i], A[i+1], \ldots, A[\mathrm{Dir}]$ são maiores ou iguais a x.

Quicksort

- Ilustração do processo de partição:

1	2	3	4	5	6
O	*R*	***D***	*E*	*N*	*A*
A	*R*	***D***	*E*	*N*	*O*
A	***D***	*R*	*E*	*N*	*O*

- O pivô x é escolhido como sendo $A[(i + j) \text{ div } 2]$.
- Como inicialmente $i = 1$ e $j = 6$, então $x = A[3] = D$.
- Ao final do processo de partição i e j se cruzam em $i = 3$ e $j = 2$.

Quicksort

Procedimento Particao:

```
void Particao(TipoIndice Esq, TipoIndice Dir, TipoIndice *i,TipoIndice *j, TipoItem *A)
{ TipoItem x, w;
  *i = Esq;  *j = Dir;
  x = A[(*i + *j) / 2]; /* obtem o pivo x */
  do
    { while (x.Chave > A[*i].Chave) (*i)++;
    while (x.Chave < A[*j].Chave) (*j)--;
    if (*i <= *j)
      { w = A[*i]; A[*i] = A[*j]; A[*j] = w; (*i)++;
       (*i)++; (*j)--;
      }
    } while (*i <= *j);
}
```

- O anel interno do procedimento Particao é extremamente simples.
- Razão pela qual o algoritmo Quicksort é tão rápido.

Quicksort

Procedimento Quicksort:

```
/*-- Entra aqui o procedimento Particao da transparencia 32
--*/
void Ordena(TipoIndice Esq, TipoIndice Dir, TipoItem *A)
{ TipoIndice i, j;
  Particao(Esq, Dir, &i, &j, A); i
  f (Esq < j) Ordena(Esq, j, A);
  if (i < Dir) Ordena(i, Dir, A);
}

void QuickSort(TipoItem *A, TipoIndice n)
{ Ordena(1, n, A); }
```

Quicksort

- Exemplo do estado do vetor em cada chamada recursiva do procedimento Ordena:

	1	2	3	4	5	6
Chaves iniciais:	*O*	*R*	*D*	*E*	*N*	*A*
1	*A*	***D***	*R*	*E*	*N*	*O*
2	***A***	*D*				
3			***E***	*R*	*N*	*O*
4				***N***	*R*	*O*
5					*O*	***R***
	A	*D*	*E*	*N*	*O*	*R*

Quicksort: Análise

- Seja $C(n)$ a função que conta o número de comparações.
- Pior caso:

$$C(n) = O(n^2)$$

- O pior caso ocorre quando, sistematicamente, o pivô é escolhido como sendo um dos extremos de um arquivo já ordenado.
- Isto faz com que o procedimento Ordena seja chamado recursivamente n vezes, eliminando apenas um item em cada chamada.
- O pior caso pode ser evitado empregando pequenas modificações no algoritmo.
- Para isso basta escolher três itens quaisquer do vetor e usar a **mediana dos três** como pivô.

Quicksort: Análise

- Melhor caso:

$$C(n) = 2C(n/2) + n = n \log n - n + 1$$

- Esta situação ocorre quando cada partição divide o arquivo em duas partes iguais.

- Caso médio de acordo com Sedgewick e Flajolet (1996, p. 17):

$$C(n) \approx 1,386n \log n - 0,846n,$$

- Isso significa que em média o tempo de execução do Quicksort é $O(n \log n)$.

Quicksort

- Vantagens:
  - É extremamente eficiente para ordenar arquivos de dados.
  - Necessita de apenas uma pequena pilha como memória auxiliar.
  - Requer cerca de $n \log n$ comparações em média para ordenar n itens.
- Desvantagens:
  - Tem um pior caso $O(n^2)$ comparações.
  - Sua implementação é muito delicada e difícil:
    - Um pequeno engano pode levar a efeitos inesperados para algumas entradas de dados.
  - O método não é **estável**.

Heapsort

- Possui o mesmo princípio de funcionamento da ordenação por seleção.
- Algoritmo:
  1. Selecione o menor item do vetor.
  2. Troque-o com o item da primeira posição do vetor.
  3. Repita estas operações com os $n - 1$ itens restantes, depois com os $n - 2$ itens, e assim sucessivamente.
- O custo para encontrar o menor (ou o maior) item entre n itens é $n - 1$ comparações.
- Isso pode ser reduzido utilizando uma fila de prioridades.

Heapsort

Filas de Prioridades

- É uma estrutura de dados onde a chave de cada item reflete sua habilidade relativa de abandonar o conjunto de itens rapidamente.

- Aplicações:
  - SOs usam filas de prioridades, nas quais as chaves representam o tempo em que eventos devem ocorrer.
  - Métodos numéricos iterativos são baseados na seleção repetida de um item com maior (menor) valor.
  - Sistemas de gerência de memória usam a técnica de substituir a página menos utilizada na memória principal por uma nova página.

Heapsort

Filas de Prioridades - Tipo Abstrato de Dados

- Operações:
  1. Constrói uma fila de prioridades a partir de um conjunto com *n* itens.
  2. Informa qual é o maior item do conjunto.
  3. Retira o item com maior chave.
  4. Insere um novo item.
  5. Aumenta o valor da chave do item *i* para um novo valor que é maior que o valor atual da chave.
  6. Substitui o maior item por um novo item, a não ser que o novo item seja maior.
  7. Altera a prioridade de um item.
  8. Remove um item qualquer.
  9. Ajunta duas filas de prioridades em uma única.

Heapsort

Filas de Prioridades - Representação

- Representação através de uma lista linear ordenada:
  - Neste caso, Constrói leva tempo $O(n \log n)$.
  - Insere é $O(n)$.
  - Retira é $O(1)$.
  - Ajunta é $O(n)$.
- Representação é através de uma lista linear não ordenada:
  - Neste caso, Constrói tem custo linear.
  - Insere é $O(1)$.
  - Retira é $O(n)$.
  - Ajunta é $O(1)$ para apontadores e $O(n)$ para arranjos.

Heapsort

Filas de Prioridades - Representação

- A melhor representação é através de uma estruturas de dados chamada *heap*:
  - Neste caso, Constrói é $O(n)$.
  - Insere, Retira, Substitui e Altera são $O(\log n)$.
- **Observação**:
  Para implementar a operação Ajunta de forma eficiente e ainda preservar um custo logarítmico para as operações Insere, Retira, Substitui e Altera é necessário utilizar estruturas de dados mais sofisticadas, tais como árvores binomiais (Vuillemin, 1978).

Heapsort

Filas de Prioridades - Algoritmos de Ordenação

- As operações das filas de prioridades podem ser utilizadas para implementar algoritmos de ordenação.
- Basta utilizar repetidamente a operação Insere para construir a fila de prioridades.
- Em seguida, utilizar repetidamente a operação Retira para receber os itens na ordem reversa.
- O uso de listas lineares não ordenadas corresponde ao método da seleção.
- O uso de listas lineares ordenadas corresponde ao método da inserção.
- O uso de *heaps* corresponde ao método Heapsort.

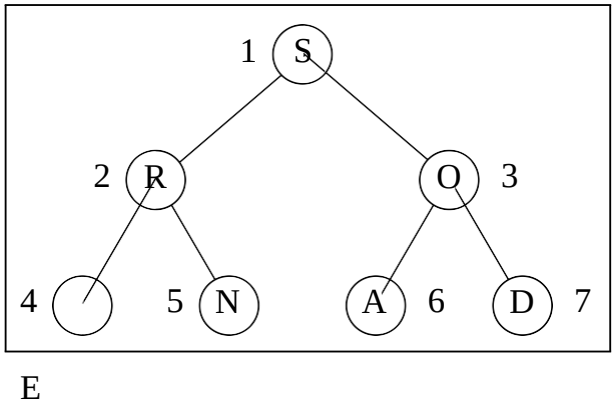
Heap

- É uma seqüência de itens com chaves $c[1], c[2], \ldots, c[n]$, tal que:

$$c[i] \geq c[2i],$$
$$c[i] \geq c[2i+1],$$

para todo $i = 1, 2, \ldots, n/2$.

- A definição pode ser facilmente visualizada em uma árvore binária completa:



Heap

- **Árvore binária completa**:
  - Os nós são numerados de 1 a n.
  - O primeiro nó é chamado raiz.
  - O nó $\lfloor k/2 \rfloor$ é o pai do nó k, para $1 < k \leq n$.
  - Os nós $2k$ e $2k + 1$ são os filhos à esquerda e à direita do nó k, para $1 \leq k \leq \lfloor k/2 \rfloor$.

Heap

- As chaves na árvore satisfazem a condição do *heap*.
- A chave em cada nó é maior do que as chaves em seus filhos.
- A chave no nó raiz é a maior chave do conjunto.
- Uma árvore binária completa pode ser representada por um array:

1	2	3	4	5	6	7
S	*R*	*O*	*E*	*N*	*A*	*D*

- A representação é extremamente compacta.
- Permite caminhar pelos nós da árvore facilmente.
- Os filhos de um nó i estão nas posições $2i$ e $2i + 1$.
- O pai de um nó i está na posição $i \operatorname{div} 2$.

Heap

- Na representação do *heap* em um arranjo, a maior chave está sempre na posição 1 do vetor.
- Os algoritmos para implementar as operações sobre o *heap* operam ao longo de um dos caminhos da árvore.
- Um algoritmo elegante para construir o *heap* foi proposto por Floyd em 1964.
- O algoritmo não necessita de nenhuma memória auxiliar.
- Dado um vetor $A[1], A[2], \ldots, A[n]$.
- Os itens $A[n/2+1], A[n/2+2], \ldots, A[n]$ formam um *heap*:
  - Neste intervalo não existem dois índices i e j tais que $j = 2i$ ou $j = 2i + 1$.

Heap

	1	2	3	4	5	6	7
Chaves iniciais:	*O*	*R*	*D*	*E*	*N*	*A*	*S*
Esq = 3	*O*	*R*	***S***	*E*	*N*	*A*	***D***
Esq = 2	*O*	*R*	*S*	*E*	*N*	*A*	*D*
Esq = 1	***S***	*R*	***O***	*E*	*N*	*A*	*D*

- Os itens de $A[4]$ a $A[7]$ formam um *heap*.
- O *heap* é estendido para a esquerda ($Esq = 3$), englobando o item $A[3]$, pai dos itens $A[6]$ e $A[7]$.

Heap

- A condição de *heap* é violada:
  - O *heap* é refeito trocando os itens D e S.
- O item R é incluindo no *heap* ($Esq = 2$), o que não viola a condição de *heap*.
- O item O é incluindo no *heap* ($Esq = 1$).
- A Condição de *heap* violada:
  - O *heap* é refeito trocando os itens O e S, encerrando o processo.

O Programa que implementa a operação que informa o item com maior chave:

```
TipoItem Max(TipoItem *A)
{ return (A[1]); }
```

Heap

Programa para refazer a condição de *heap*:

```
void Refaz(TipoIndice Esq, TipoIndice Dir, TipoItem *A)
{ TipoIndice i = Esq;
  int j; TipoItem x; j
  = i * 2;
  x = A[i];
  while (j <= Dir)
    { if (j < Dir)
      { if (A[j].Chave < A[j+1].Chave)
        j++;
      }
      if (x.Chave >= A[j].Chave) goto L999;
      A[i] = A[j]; i = j;  j = i * 2
    }                      ;
  L999: A[i] = x;
}
```

Heap

Programa para construir o *heap*:

```
void Constroi(TipoItem        TipoIndice n)
*A,
{ TipoIndice Esq;
  Esq = n / 2 + 1;
  while (Esq > 1)
    { Esq--;
      Refaz(Esq, n, A);
    }
}
```

Heap

Programa que implementa a operação de retirar o item com maior chave:

```
TipoItem RetiraMax(TipoItem *A, TipoIndice *n)
{ TipoItem Maximo;
  if (*n < 1)
  printf("Erro: heap vazio\n");
  else { Maximo = A[1];  A[1] = A[*n];
                         (*n)--; Refaz(1, *n,
         A);
       }
  return Maximo;
}
```

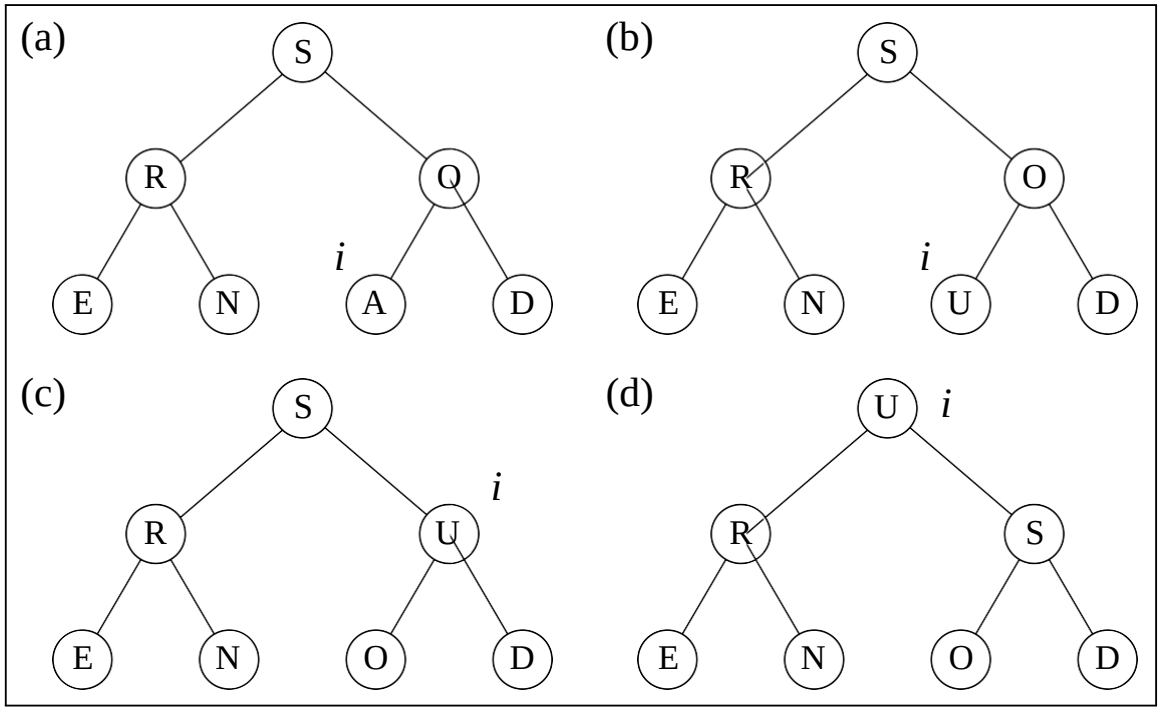
Heap

Programa que implementa a operação de aumentar o valor da chave do item *i*:

```
void AumentaChave(TipoIndice i, TipoChave ChaveNova, TipoItem *A)
{ TipoItem x;
  if (ChaveNova < A[i].Chave)
  { printf("Erro: ChaveNova menor que a chave atual\n");
    return;
  }
  A[i].Chave = ChaveNova;
  while (i > 1 && A[i / 2].Chave < A[i].Chave)
    { x = A[i / 2]; A[i / 2] = A[i]; A[i] = x;
      i /= 2;
    }
}
```

Heap

- Exemplo da operação de aumentar o valor da chave do item na posição *i*:



- O tempo de execução do procedimento AumentaChave em um item do *heap* é $O(\log n)$.

Heap

Programa que implementa a operação de inserir um novo item no *heap*:

```
void Insere(TipoItem *x, TipoItem *A, TipoIndice *n)
{(*n)++; A[*n] = *x; A[*n].Chave =
  INT_MIN; AumentaChave(*n, x->Chave, A);
}
```

Heapsort

- Algoritmo:
  1. Construir o *heap*.
  2. Troque o item na posição 1 do vetor (raiz do *heap*) com o item da posição n.
  3. Use o procedimento Refaz para reconstituir o *heap* para os itens $A[1], A[2], \ldots, A[n-1]$.
  4. Repita os passos 2 e 3 com os $n-1$ itens restantes, depois com os $n-2$, até que reste apenas um item.

Heapsort

- Exemplo de aplicação do Heapsort:

1	2	3	4	5	6	7
S	*R*	*O*	*E*	*N*	*A*	*D*
R	***N***	*O*	*E*	***D***	*A*	***S***
O	*N*	***A***	*E*	*D*	*R*	
N	***E***	*A*	***D***	*O*		
E	***D***	*A*	*N*			
D	***A***	*E*				
A	*D*					

- O caminho seguido pelo procedimento Refaz para reconstituir a condição do *heap* está em negrito.
- Por exemplo, após a troca dos itens S e D na segunda linha da Figura, o item D volta para a posição 5, após passar pelas posições 1 e 2.

Heapsort

Programa que mostra a implementação do Heapsort:

```
void Heapsort(TipoItem *A, TipoIndice n)
{ TipoIndice Esq, Dir;
  TipoItem x;
  Constroi(A, n);  /* constroi o heap */
  Esq = 1;  Dir = n;
  while (Dir > 1)
    { /* ordena o vetor */
      x = A[1];  A[1] = A[Dir];  A[Dir] = x;  Dir--;
      Refaz(Esq, Dir, A);
    }
}
```

Análise

- O procedimento Refaz gasta cerca de $\log n$ operações, no pior caso.

- Logo, Heapsort gasta um tempo de execução proporcional a $n \log n$, no pior caso.

Heapsort

- Vantagens:
  - O comportamento do Heapsort é sempre $O(n \log n)$, qualquer que seja a entrada.
- Desvantagens:
  - O anel interno do algoritmo é bastante complexo se comparado com o do Quicksort.
  - O Heapsort não é **estável**.
- Recomendado:
  - Para aplicações que não podem tolerar eventualmente um caso desfavorável.
  - Não é recomendado para arquivos com poucos registros, por causa do tempo necessário para construir o *heap*.

Comparação entre os Métodos

Complexidade:

	Complexidade
Inserção	$O(n^2)$
Seleção	$O(n^2)$
Shellsort	$O(n \log n)$
Quicksort	$O(n \log n)$
Heapsort	$O(n \log n)$

- Apesar de não se conhecer analiticamente o comportamento do Shellsort, ele é considerado um método eficiente).

Comparação entre os Métodos

Tempo de execução:

- Oservação: O método que levou menos tempo real para executar recebeu o valor 1 e os outros receberam valores relativos a ele.

- Registros na ordem aleatória:

	5.00	5.000	10.000	30.000
Inserção	11,3	87	161	–
Seleção	16,2	124	228	–
Shellsort	1,2	1,6	1,7	2
Quicksort	1	1	1	1
Heapsort	1,5	1,6	1,6	1,6

Comparação entre os Métodos

- Registros na ordem ascendente:

	500	5.000	10.000	30.000
Inserção	1	1	1	1
Seleção	128	1.524	3.066	–
Shellsort	3,9	6,8	7,3	8,1
Quicksort	4,1	6,3	6,8	7,1
Heapsort	12,2	20,8	22,4	24,6

Comparação entre os Métodos

Tempo de execução:

- Registros na ordem descendente:

	500	5.000	10.000	30.000
Inserção	40,3	305	575	–
Seleção	29,3	221	417	–
Shellsort	1,5	1,5	1,6	1,6
Quicksort	1	1	1	1
Heapsort	2,5	2,7	2,7	2,9

Comparação entre os Métodos

Observações sobre os métodos:

1. Shellsort, Quicksort e Heapsort têm a mesma ordem de grandeza.
2. O Quicksort é o mais rápido para todos os tamanhos aleatórios experimentados.
3. A relação Heapsort/Quicksort é constante para todos os tamanhos.
4. A relação Shellsort/Quicksort aumenta se o número de elementos aumenta.
5. Para arquivos pequenos (500 elementos), o Shellsort é mais rápido que o Heapsort.
6. Se a entrada aumenta, o Heapsort é mais rápido que o Shellsort.
7. O Inserção é o mais rápido se os elementos estão ordenados.
8. O Inserção é o mais lento para qualquer tamanho se os elementos estão em ordem descendente.
9. Entre os algoritmos de custo $O(n^2)$, o Inserção é melhor para todos os tamanhos aleatórios experimentados.

Comparação entre os Métodos

Influência da ordem inicial dos registros:

	Shellsort			Quicksort			Heapsort		
	5.000	10.000	30.000	5.000	10.000	30.000	5.000	10.000	30.000
Asc	1	1	1	1	1	1	1,1	1,1	1,1
Des	1,5	1,6	1,5	1,1	1,1	1,1	1	1	1
Ale	2,9	3,1	3,7	1,9	2,0	2,0	1,1	1	1

1. Shellsort é bastante sensível à ordenação ascendente ou descendente da entrada.
2. Em arquivos do mesmo tamanho, o Shellsort executa mais rápido para arquivos ordenados.
3. Quicksort é sensível à ordenação ascendente ou descendente da entrada.
4. Em arquivos do mesmo tamanho, o Quicksort executa mais rápido para arquivos ordenados.
5. O Quicksort é o mais rápido para qualquer tamanho para arquivos na ordem ascendente.
6. O Heapsort praticamente não é sensível à ordenação da entrada.

Comparação entre os Métodos

Método da Inserção:

- É o mais interessante para arquivos com menos do que 20 elementos.
- O método é estável.
- Possui comportamento melhor do que o método da **bolha (Bubblesort)** que também é estável.
- Sua implementação é tão simples quanto as implementações do Bubblesort e Seleção.
- Para arquivos já ordenados, o método é $O(n)$.
- O custo é linear para adicionar alguns elementos a um arquivo já ordenado.

Comparação entre os Métodos

Método da Seleção:

- É vantajoso quanto ao número de movimentos de registros, que é $O(n)$.
- Deve ser usado para arquivos com registros muito grandes, desde que o tamanho do arquivo não exceda 1.000 elementos.

Comparação entre os Métodos

Shellsort:

- É o método a ser escolhido para a maioria das aplicações por ser muito eficiente para arquivos de tamanho moderado.
- Mesmo para arquivos grandes, o método é cerca de apenas duas vezes mais lento do que o Quicksort.
- Sua implementação é simples e geralmente resulta em um programa pequeno.
- Não possui um pior caso ruim e quando encontra um arquivo parcialmente ordenado trabalha menos.

Comparação entre os Métodos

Quicksort:

- É o algoritmo mais eficiente que existe para uma grande variedade de situações.
- É um método bastante frágil no sentido de que qualquer erro de implementação pode ser difícil de ser detectado.
- O algoritmo é recursivo, o que demanda uma pequena quantidade de memória adicional.
- Seu desempenho é da ordem de $O(n^2)$ operações no pior caso.
- O principal cuidado a ser tomado é com relação à escolha do pivô.
- A escolha do elemento do meio do arranjo melhora muito o desempenho quando o arquivo está total ou parcialmente ordenado.
- O pior caso tem uma probabilidade muito remota de ocorrer quando os elementos forem aleatórios.

Comparação entre os Métodos

Quicksort:

- Geralmente se usa a mediana de uma amostra de três elementos para evitar o pior caso.
- Esta solução melhora o caso médio ligeiramente.
- Outra importante melhoria para o desempenho do Quicksort é evitar chamadas recursivas para pequenos subarquivos.
- Para isto, basta chamar um método de ordenação simples nos arquivos pequenos.
- A melhoria no desempenho é significativa, podendo chegar a 20% para a maioria das aplicações (Sedgewick, 1988).

Comparação entre os Métodos

Heapsort:

- É um método de ordenação elegante e eficiente.
- Apesar de ser cerca de duas vezes mais lento do que o Quicksort, não necessita de nenhuma memória adicional.
- Executa sempre em tempo proporcional a $n \log n$,
- Aplicações que não podem tolerar eventuais variações no tempo esperado de execução devem usar o Heapsort.

Comparação entre os Métodos

Considerações finais:

- Para registros muito grandes é desejável que o método de ordenação realize apenas n movimentos dos registros.
- Com o uso de uma **ordenação indireta** é possível se conseguir isso.
- Suponha que o arquivo A contenha os seguintes registros: $A[1], A[2], \ldots, A[n]$.
- Seja P um arranjo $P[1], P[2], \ldots, P[n]$ de apontadores.
- Os registros somente são acessados para fins de comparações e toda movimentação é realizada sobre os apontadores.
- Ao final, $P[1]$ contém o índice do menor elemento de A, $P[2]$ o índice do segundo menor e assim sucessivamente.
- Essa estratégia pode ser utilizada para qualquer dos métodos de ordenação interna.